



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

University of Patras
Department of Mechanical Engineering and Aeronautics

Wing Structural Dynamics

Project in the context of the course
“Dynamics of Aerospace Structures”

Academic Year 2025-2026

Table of Contents

1	Theoretical Background.....	2
1.1	Standard Eigenvalue Problem	2
1.2	Mode orthogonality and modal matrices.....	4
1.3	Modal space	5
1.4	Modal analysis – Forced vibration of undamped system.....	5
1.5	Solution of 2 nd -order O.D.Es for damped and undamped systems.....	7
1.5.1	Undamped system	7
1.5.2	Damped system.....	8
1.6	Proportional Damping and Complex Eigensolution	8
1.7	Numerical Integration.....	10
1.7.1	Central differences.....	10
1.7.2	Newmark method	11
2	Task presentation and results	12
2.1	Calculation of the system's natural frequencies and presentation of the first five eigenmodes.....	12
2.2	Solution in the modal space for an applied force with two different forcing frequencies.....	15
2.2.1	First Part.....	15
2.2.2	Second Part	19
2.3	Proportional Reyleigh Damping	19
2.4	Response of damped system via the complex eigensolution	24
2.5	Transient response with the use of explicit numerical integration.....	25
2.6	Transient response with the use of explicit numerical integration and generalized/modal matrices.....	27
2.7	Aeroelasticity.....	29
3	Bibliography	35
4	Appendix.....	36

1 Theoretical Background

This section presents the theoretical framework underlying the fundamental aspects of dynamic modelling and methodologies employed in this project. Firstly, the eigenvalue problem and the use of modal space are introduced, followed by a discussion regarding the concepts of mode orthogonality, expansion theorem, as well as mode superposition. Furthermore, proportional damping and complex eigensolution are presented. Solutions to 2nd-order differential equations for systems with and without damping are briefly discussed. Lastly, numerical integration schemes such as central differences and Newmark method are shown, with emphasis on the steps needed for their implementation.

1.1 Standard Eigenvalue Problem

It is known that for any system, the equations of motion can be expressed in matrix form as

$$[m]\ddot{\vec{x}} + [c]\dot{\vec{x}} + [k]\vec{x} = \vec{F} \quad (1)$$

where $[m]$, $[c]$ and $[k]$ are the mass, damping and stiffness matrices of the system, respectively. The vectors $\ddot{\vec{x}}$, $\dot{\vec{x}}$, $\vec{x}$ represent the system's acceleration, velocity and displacement, whereas the vector $\vec{F}$ are the forces acting on the system. The matrices $[m]$, $[c]$, $[k]$ can be either diagonal or non-diagonal (non-zero off-diagonal terms). In the case of non-diagonal matrices, the system displays mass, damping and elastic coupling and the equations are coupled. A system that has n degrees of freedom requires n independent coordinates in order to describe its motion sufficiently. If coupling is present in the system, the equations of motion are difficult to solve in the physical coordinates because they are not independent, thus a different coordinate system needs to be adopted. For this reason, a different approach is followed where a different coordinate system is selected whose coordinates are called *generalized coordinates*.

When the system is undamped and no forces are acting upon it, then equation (1) is reduced to

$$[m]\ddot{\vec{x}} + [k]\vec{x} = 0 \quad (2)$$

The solution of (2) is given if a solution of the form is assumed

$$x_i(t) = \Phi_i T(t) \quad , i = 1, 2 \dots n \text{ Dof} \quad (3)$$

where Φ_i is a time-independent vector called the *mode shape of the system* and $T(t)$ is a function of time.

If equation (3) is substituted in equation (2), the problem can be expressed as

$$[m]\ddot{\vec{\Phi}}T(t) + [k]\vec{\Phi}T(t) = \vec{0} \quad (4)$$

and rewritten in the forms of equation (5) and (6)

$$T''(t) + \omega^2 T(t) = 0 \quad (5)$$

$$[[k] - \omega^2[m]]\vec{\Phi} = \vec{0} \quad (6)$$

where ω^2 is initially set as a constant and later serves as the frequency for the harmonic motion $T(t)$ derived from equation (5). The values of ω are specific and need to be calculated. Since, equation (6) represents a set of n equations, then n ω^2 terms can be derived from the enforcement of zero determinant (equation 7).

$$\text{Det} = |[k] - \omega^2[m]| = 0 \quad (7)$$

Equation (7) is called the *characteristic equation* that yields n solutions (equal to the degrees of freedom) of ω_i^2 which are the eigenvalues of the system. The values ω_i correspond to each natural frequency the system has.

The problem can also be re-expressed with the use of $\lambda = \frac{1}{\omega^2}$ as

$$[\lambda[k] - [m]]\vec{\Phi} = \vec{0} \quad (8)$$

And equivalently by multiplying with the inverse of the stiffness matrix $[k]^{-1}$

$$\lambda[I]\vec{\Phi} = [D]\vec{\Phi} \quad (9)$$

where $[I]$ is the unity matrix and $[D]$ is the dynamical matrix which is equal to

$$[D] = [k]^{-1}[m] \quad (10)$$

Equation (9) is called the *standard eigenvalue problem*. By enforcing the determinant to be zero, the eigenvalues are computed. If the eigenvalues λ_i are substituted in equation (9) then the mode shapes Φ_i can also be found, otherwise called eigenvectors. The term eigenmode is basically equivalent to the term eigenvector, however when referred to as eigenmode, it is to showcase the pair of a natural frequency and its corresponding mode shape. It is understood that mode shapes describe the pattern of the system's displacement under a vibration that occurs from a specific natural frequency.

1.2 Mode orthogonality and modal matrices

One of the most important properties of mode shapes is that they are inherently orthogonal. Equations (11) and (12) indicate that the mode shapes or eigenvectors are orthogonal with respect to the mass and stiffness matrices

$$\vec{\Phi}^{(j)T}[m]\vec{\Phi}^{(i)} = 0, \quad i \neq j \quad (11)$$

$$\vec{\Phi}^{(j)T}[k]\vec{\Phi}^{(i)} = 0, \quad i \neq j \quad (12)$$

When $i = j$, equations (11) and (12) give the generalized mass and stiffness coefficients correspond to the i th mode.

$$M_{ii} = \vec{\Phi}^{(i)T}[m]\vec{\Phi}^{(i)}, \quad i = 1, 2, \dots, n \quad (13)$$

$$K_{ii} = \vec{\Phi}^{(i)T}[k]\vec{\Phi}^{(i)}, \quad i = 1, 2, \dots, n \quad (14)$$

The generalized mass and stiffness coefficients are elements of the generalized or modal mass and stiffness matrices, as shown in (15) and (16).

$$[M_g] = \begin{bmatrix} M_{11} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & M_{nn} \end{bmatrix} = [\Phi]^T[m][\Phi] \quad (15)$$

$$[K_g] = \begin{bmatrix} K_{11} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & K_{nn} \end{bmatrix} = [\Phi]^T[k][\Phi] \quad (16)$$

The matrix $[\Phi]$ is also called the modal matrix and it contains all of the eigenvectors-mode shapes calculated from the standard eigenvalue problem. The modal matrix's columns correspond to the eigenvectors and the rows to the degrees of freedom. It is essential to note that the generalized matrices are inherently diagonal.

In many cases, the mode shapes are manipulated in order to be normalized with respect to the mass matrix. The normalization is implemented with equation (17)

$$\vec{\Phi}^{(j)T}[m]\vec{\Phi}^{(i)} = 1, \quad i = 1, 2, \dots, n \quad (17)$$

From equation (17), the generalized mass matrix reduces to the unity $[I]$ matrix and the generalized stiffness matrix transform to (18) where the diagonal elements are eigenvalues.

$$[K_g] = \begin{bmatrix} \omega_1^2 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \omega_n^2 \end{bmatrix} \quad (18)$$

When mode shapes Φ are both orthogonal and normalized with respect to the mass matrix, they are called *orthonormal* with respect to the mass matrix.

1.3 Modal space

If the system's matrices are non-diagonal, then solutions are difficult to derive, and the use of a set of generalized coordinates is preferred. The standard eigenvalue problem yields n mode shapes Φ_i that are linearly independent from each other, therefore they form a basis in the n -dimensional space. Consequently, any vector of the n -dimensional space can be expressed as a linear combination of the n -linearly independent eigenvectors. In essence, a coordinate transformation is possible by utilizing the eigenvectors derived from the standard eigenvalue problem. The space that is described with the use of eigenvectors is called *modal space* and the transformation from the physical space to the modal space is expressed as

$$\vec{x} = \sum_{i=1}^n q_i(t) \vec{\Phi}^{(i)} \quad (19)$$

where the right hand side is the physical space solution that is computed via linear combination of the mode shapes. The terms $q_i(t)$ are time-dependent constants called modal coordinates that essentially dictate the contribution of each mode shape. These terms are also called *modal participation coefficients*.

1.4 Modal analysis – Forced vibration of undamped system

The ability to derive the solution with the utilization of modal space is very convenient compared to solving directly in physical space. For the case of an undamped system with forced vibration, the equations of motion are expressed in matrix form as

$$[m]\ddot{\vec{x}} + [k]\vec{x} = \vec{F} \quad (20)$$

As mentioned previously, the expression in the physical space is comprised of n coupled 2nd-order ordinary differential equations. Because the solution of such equation set is complex, the system is rewritten in the modal space. The process of mapping the physical space solution to the modal space solution is called the *modal analysis*. The main advantage of modal analysis is the use of the generalized matrices, which are inherently diagonal. As

a result, the equation set is now comprised of n uncoupled homogeneous linear 2nd-order ordinary differential equations.

The modal analysis begins by forming and solving the standard eigenvalue problem (equation 6). In this way, the natural frequencies ω_i^2 are derived from its determinant and then used to derive the mode shapes Φ_i . Equation (19) is expanded to map the physical space to the modal space as shown in (21).

$$\vec{x} = q_1(t)\vec{\Phi}^{(1)} + q_2(t)\vec{\Phi}^{(2)} + \dots + q_n(t)\vec{\Phi}^{(n)} \quad (21)$$

Equation (21) can be expressed as

$$\vec{x}(t) = [\Phi]\vec{q}(t) \quad (22)$$

$$\text{where the modal matrix is } [\Phi] = [\vec{\Phi}^{(1)} \ \vec{\Phi}^{(2)} \ \dots \ \vec{\Phi}^{(n)}] \quad (23)$$

and the modal participation coefficients are

$$\vec{q}(t) = \begin{Bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{Bmatrix} \quad (24)$$

Equation (22) can be differentiated and replaced in the initial system expression (20). If the mode shapes are orthonormal, then equation (25) is obtained

$$[Mg]\ddot{\vec{q}} + [Kg]\vec{q} = [\Phi]^T \vec{F} \rightarrow [I]\ddot{\vec{q}} + [\omega_i^2]\vec{q} = \vec{Q}(t) \quad (25)$$

where $\vec{Q}(t)$ is the vector of generalized forces (expressed in the modal space).

The expression of equation (25) is a set of n decoupled 2nd-order O.D.Es whose solutions are obtained with conventional methodologies and describe n time-dependent vectors $\vec{q}_i(t)$ corresponding to n natural frequencies ω . The solution of the ODEs is presented in 1.5 . Before solving the O.D.Es, equation (22) is going to be used to transform the physical space initial conditions to modal space initial conditions.

At this point , it is valuable to present the idea of mode displacement method where instead of finding the physical space solution from the linear combination of n modes , a reduced amount of modes is used. The reasoning of this method is to sufficiently approximate the solution with a substantially reduced amount of modes , thus lowering complexity and potential computational costs(since these problems are solved with the numerical methods). Despite using a reduced amount of modes , the physical space solution is approximated adequately because , for most systems, the first natural frequencies have the highest energy content.

1.5 Solution of 2nd-order O.D.Es for damped and undamped systems

Modal analysis requires the solution of the modal participation coefficients which are essentially 2nd-order O.D.Es. These O.D.Es have specific solutions which depend on the initial conditions and natural frequency. There are two distinct cases that will be presented, the former being the undamped system (equation 20) and the latter being the damped system (equation 1). The external force is assumed to be a harmonic vibration with forcing frequency ω_f as shown in (25).

$$F(t) = F \sin(\omega_f t) \quad (26)$$

The system's initial conditions are an initial displacement $\vec{q}(0)$ and an initial velocity $\dot{\vec{q}}(0)$.

It must be noted that the solutions are derived with the assumption that modes shapes are orthonormal with respect to the mass matrix.

1.5.1 Undamped system

The fundamental solution of the damped system consists from the addition of the complementary (26) and particular (27) solutions, which correspond to the homogeneous and non-homogenous respectively.

$$u_c(t) = A \cos(\omega_n t) + B \sin(\omega_n t) \quad (27)$$

$$u_p(t) = C \sin(\omega_f t) \quad (28)$$

Thus the combined solution is

$$u(t) = u_c(t) + u_p(t) = A \cos(\omega_n t) + B \sin(\omega_n t) + C \sin(\omega_f t) \quad (29)$$

where A,B,C are coefficients that are obtained with the use of initial conditions and are presented in (30)

$$\left\{ \begin{array}{l} A = \vec{q}(0) \\ B = \frac{\dot{\vec{q}}(0)}{\omega_n} - \frac{F_o}{\omega_n^2 - \omega_f^2} \\ C = \frac{F_o}{\omega_n^2 - \omega_f^2} \end{array} \right\} \quad (30)$$

1.5.2 Damped system

Similarly with the former solution, the complementary and particular solutions are shown in (31) and (32).

$$u_c(t) = e^{-\zeta\omega_n t} (A \cos(\omega_d t) + B \sin(\omega_d t)) \quad (31)$$

$$u_p(t) = C \cos(\omega_f t) + D \sin(\omega_f t) \quad (32)$$

$$u(t) = e^{-\zeta\omega_n t} (A \cos(\omega_d t) + B \sin(\omega_d t)) + C \cos(\omega_f t) + D \sin(\omega_f t) \quad (33)$$

where $\omega_d = \omega_n \sqrt{1 - \zeta^2}$ is the damping frequency and ζ is the damping ratio.

The constants A,B,C,D are derived with the use of the initial conditions and have the form of (33). It must be noted that the constants have this form when initial conditions are zero.

$$\left\{ \begin{array}{l} A = -D \\ B = \frac{\zeta\omega_n A - C\omega_n}{\omega_d} \\ C = \frac{Fo}{\omega_n^2} \cdot \frac{1 - (\omega_f/\omega_n)^2}{[1 - (\omega_f/\omega_n)^2]^2 + [2\zeta(\omega_f/\omega_n)]^2} \\ D = \frac{Fo}{\omega_n^2} \cdot \frac{-2\zeta(\omega_f/\omega_n)}{[1 - (\omega_f/\omega_n)^2]^2 + [2\zeta(\omega_f/\omega_n)]^2} \end{array} \right\} \quad (34)$$

1.6 Proportional Damping and Complex Eigensolution

In the case where the system is non-conservative (damped), modal analysis cannot be directly implemented unless damping is very light. In many cases, the introduction of damping leads to the appearance of modes that cannot be computed via the modal analysis. The first approach to damped systems is assuming that damping matrix [c] is proportional to the mass matrix [m] and stiffness matrix [k] (35) .

$$[c] = \alpha[m] + \beta[k] \quad (35)$$

If modal mass and modal stiffness matrices are substituted in equation (35), then the modal damping matrix is derived and can be replaced in the matrix form of the equations of motion in the modal space. Since modal damping is proportional to modal mass and

stiffness, then the process describes in sub-chapter 1.4 can be implemented, thus deriving the solution in the modal space and mapping to the physical space. The modal participation coefficients are computed with the use of the fundamental O.D.E solution (33) where each damping ratio ζ_i is found from

$$\alpha + \beta \omega_i^2 = 2\zeta_i \omega_i \quad (36)$$

Proportional damping is a sufficient approximation to the physical solution, however that is true only when the damping is light and when the damping matrix is symmetric (which in many cases is not true). In order to fully capture the motion of the system, a different approach is used called complex eigensolution.

Complex eigensolution requires the recasting the system of equations of motion into a first order system in the state space. As a result, the state vector will have double the size of physical solution vector, with the former n state variables being displacements and the latter n state variables being velocities. The system in state space is expressed as

$$\dot{\vec{x}}(t) = A\vec{x}(t) + B\vec{F}(t) \quad (37)$$

where $x(t)$ is the state vector, A is a matrix of size $2n \times 2n$ and B the load matrix of size $2n \times 2n$. Matrix A is shown in (38) and is non-symmetric.

$$A = \begin{bmatrix} 0 & [I] \\ -[M]^{-1}[K] & -[M]^{-1}[C] \end{bmatrix} \quad (38)$$

If external forces are zero, then the solution is only the free vibration caused by the system's initial conditions. For the case the solution is derived with the process shown below. The eigenvalue problem is crafted for the system and has the form

$$[A]\vec{X}(t) = \lambda\vec{X}(t) \quad (39)$$

From equation 39 there are $2n$ eigenvalues which are obtained. Consequently, the system has $2n$ eigenvectors (X) which are called the right eigenvectors. Because the matrix is non-symmetric the right eigenvectors can not form a basis, as described in sub-chapter 1.3. For this reason, left eigenvectors must be derived. In essence, left eigenvectors (Y) are the eigenvectors that are found from the adjoint problem (eigenvalues are the same for both cases).

$$[A]^T \vec{Y}(t) = \lambda \vec{Y}(t) \quad (40)$$

where $\vec{y}(t)$ are the left eigenvectors.

The left and right eigenvectors have an inherent property of being bi-orthonormal, which means they are orthonormal with respect to each other and with respect to the matrix A . Thus, a similar approach to the one of sub-chapter 1.4 can be followed where the equation system can be decoupled.

By utilizing the biorthonormality property, the solution of the system can be derived as

$$\vec{x}(t) = [X]e^{\Lambda(t-t_0)}[Y]^T \vec{x}(t_0) \quad (41)$$

where $[X]$ and $[Y]$ are matrices containing right and left eigenvectors as columns, respectively. The matrix Λ is a diagonal matrix where the diagonal elements are eigenvalues and the vector $\vec{x}(t_0)$ is the vector of the system's initial displacement.

The right hand side of equation (41) is real, thus the left hand side must also be really. Consequently, it is understood that the imaginary parts of the matrices X, Y and Λ are cancelled. The solution of (41) is the initially defined state vector from which system displacement can be derived.

1.7 Numerical Integration

A different approach to deriving the solution of the set of equations of motion (equation 1) is to use numerical integration methods. The derivatives are approximated with finite differences based on values at discrete time steps. Depending on the formulation they can either be explicit or implicit. The difference between explicit and implicit methods is that implicit numerical schemes include terms corresponding to the next time step that has not yet been calculated. The method of central differences is an example of explicit numerical scheme, whereas the Newmark method is implicit.

1.7.1 Central differences

Equation 1 can be rewritten if the derivatives are approximated with central differences. The equation then becomes

$$[m] \frac{1}{\Delta t^2} (\vec{x}_{i+1} - 2\vec{x}_i + \vec{x}_{i-1}) + [c] \frac{1}{2\Delta t} (\vec{x}_{i+1} - \vec{x}_{i-1}) + [k]\vec{x}_i = \vec{F}_i, \quad i = 1, 2, \dots, n \quad (42)$$

Where i corresponds to the current time step, $i - 1$ to the previous and $i + 1$ to the next step. It is apparent that this is an explicit numerical scheme since the only unknown is the solution at the next time step $\vec{x}_{i+1}$. Since the numerical scheme starts from $i=1$, then the values $\vec{x}_1$, $\vec{x}_0$ and $\vec{x}_{-1}$, as well as $\ddot{\vec{x}}_0$ need to be known. These values can be computed with the combination of initial conditions, equation (1) and the central differences expression for the second derivate (acceleration). Subsequently, a time step Δt must be selected and equation (43) needs to be evaluated at each step.

$$\vec{x}_{i+1} = \left[\frac{1}{\Delta t^2} [m] + [c] \frac{1}{2\Delta t} \right]^{-1} \left[\vec{F}_i - \left([k] - \frac{2}{\Delta t^2} [m] \right) \vec{x}_i - \left(\frac{1}{\Delta t^2} [m] - [c] \frac{1}{2\Delta t} \right) \vec{x}_{i-1} \right] \quad (43)$$

It must be noted that the time step selection is an essential part of the process and if not selected carefully it can “destroy” the numerical scheme. The condition that must be satisfied in order for the numerical scheme to be stable is

$$\Delta t < \Delta t_{crit} \quad , \quad \Delta t_{crit} = \frac{T_n}{pi} \quad (44)$$

where T_n is the smallest period displayed in the system (corresponding to the highest natural frequency).

1.7.2 Newmark method

The Newmark method is an implicit numerical integration scheme that requires the calculation at the next step of not only displacement but also velocity and acceleration. The numerical scheme is presented in (45).

$$\begin{aligned} \vec{x}_{i+1} = & \left[\frac{1}{\alpha \Delta t^2} [m] + \frac{\delta}{\alpha \Delta t} [c] + [k] \right]^{-1} \\ & \times \left[\vec{F}_{i+1} + [m] \left(\frac{1}{\alpha \Delta t^2} \vec{x}_i + \frac{1}{\alpha \Delta t} \dot{\vec{x}}_i + \left(\frac{1}{2\alpha} - 1 \right) \ddot{\vec{x}}_i \right) + [c] \left(\frac{\delta}{\alpha \Delta t} \vec{x}_i \right. \right. \\ & \left. \left. + \left(\frac{\delta}{\alpha} - 1 \right) \dot{\vec{x}}_i + \left(\frac{\delta}{\alpha} - 2 \right) \frac{\Delta t}{2} \ddot{\vec{x}}_i \right] \right] \quad (45) \end{aligned}$$

$$\ddot{\vec{x}}_{i+1} = \frac{1}{\alpha \Delta t^2} (\vec{x}_{i+1} - \vec{x}_i) - \frac{1}{\alpha \Delta t} \dot{\vec{x}}_i - \left(\frac{1}{2\alpha} - 1 \right) \ddot{\vec{x}}_i \quad (46)$$

$$\dot{\vec{x}}_{i+1} = \dot{\vec{x}}_i + (1 - \delta) \Delta t \ddot{\vec{x}}_i + \delta \Delta t \ddot{\vec{x}}_{i+1} \quad (47)$$

Equations 45,46 and 47 need to be evaluated at each time step. The constants δ, α are selected and their value is important for achieving numerical stability. If $\delta = \frac{1}{2}$ and $\alpha = \frac{1}{4}$ then the numerical scheme is stable for any selection of time step Δt .

2 Task presentation and results

2.1 Calculation of the system's natural frequencies and presentation of the first five eigenmodes

In order to calculate the eigenvalues and eigenvectors in MATLAB we had to create the total matrices of mass and stiffness by combining with a suitable way the elementary matrices of stiffness and mass that were given in the project's description (appendix). The user can select the number of elements which describe the total geometry by adjusting the variable FE.N. In our case, as the project requires the right number of elements so that the 10 first eigenfrequencies converge, we investigated and proved that the number of elements that are required are 120 elements. So, the dimensions of the K,M matrices became 363x363.

The dimensions of the matrices arose by multiplying the number of nodes with the Degrees of Freedom which in our case are:

$$Nodes = Elements + 1 \quad (48)$$

$$DoFs = 3 \quad (49)$$

$$Size = DoFs * Nodes \quad (50)$$

After the calculation of the matrices [K] and [M] we used the *characteristic equation* (equation 7) to calculate the ω^2 that constitute the eigenvalues of the problem. In MATLAB the eigenvalue problem can be solved with the command eig(K,M) which gives as output the eigenvalues and eigenvectors of the problem. It is important to mention that the K, M that were used in the command, have been modified and that the nodes of the support have not been taken into consideration (we erased the first three rows and columns of the matrices that describe the first 3 DoFs-first node). The exact function call-up that was used was [X,natural_freq,lamda] = solve_eigenvalue_problem(K,M) where X (size 360x360) are the eigenvectors, natural_freq (size 360x1) are the natural frequencies and lamda (size 360x1) are the eigenmodes. Hence, the omegas(ω_i) were solved by finding the square root of the lamda.

At this point we plotted only the first five eigenvectors by selecting the first five columns of the modal matrix X (figures 1-5) , while the five first natural frequencies can be shown in *Table 1*:

f_1	3.22 Hz
f_2	20.18 Hz
f_3	25.88 Hz
f_4	56.41 Hz
f_5	77.64 Hz

Table 1: The five first natural frequencies of the wing.

Starting from the first row, the displacement w can be found with a step of 3 , the torsion bx with step of 1 and the bending by with step of 2. The five first eigenvectors can be shown in figures 1 to 5.

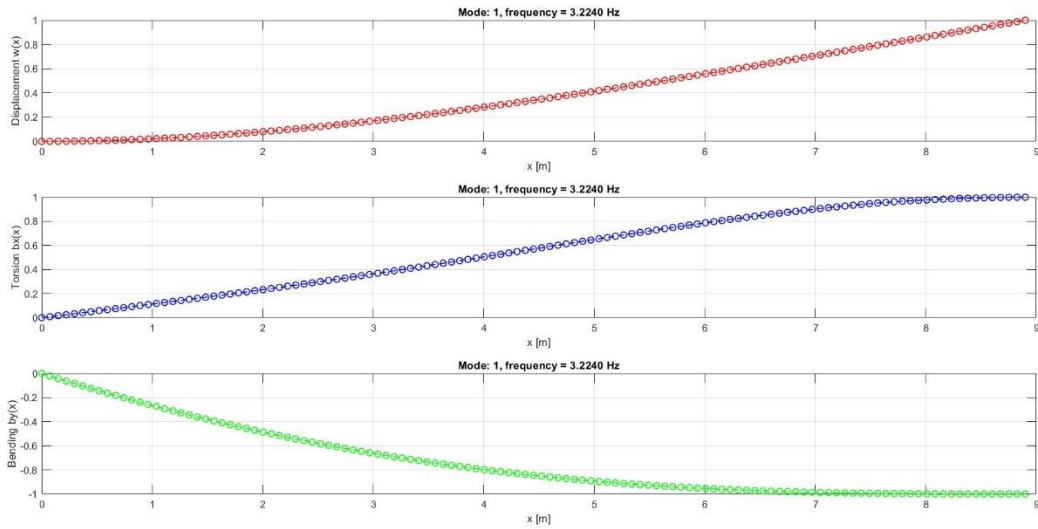


Figure 1. 1st eigenmode for 3.22Hz / $w(x)$ -displacement, $bx(x)$ Torsion, $by(x)$ Bending.

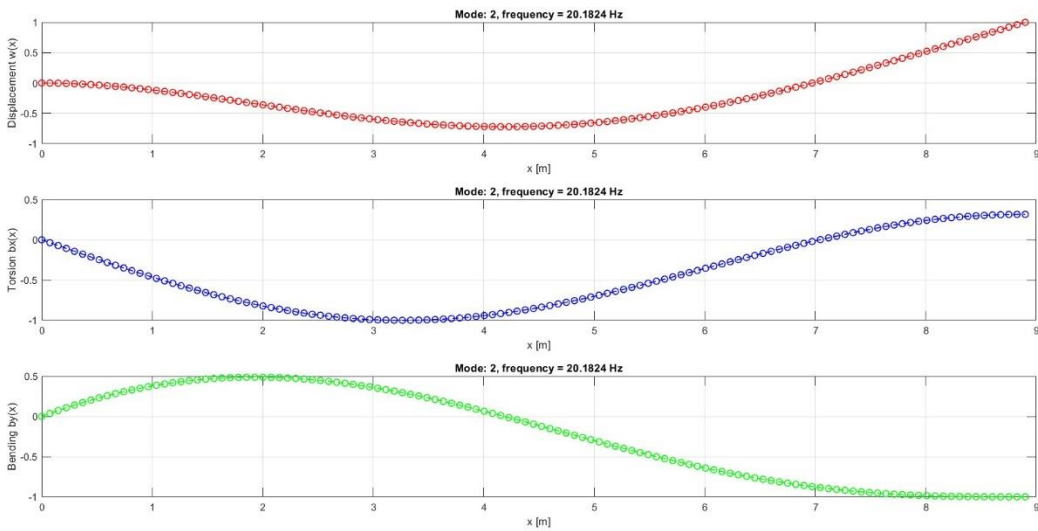


Figure 2. 2nd eigenmode for 20,18Hz/ $w(x)$ -displacement, $bx(x)$ Torsion, $by(x)$ Bending.

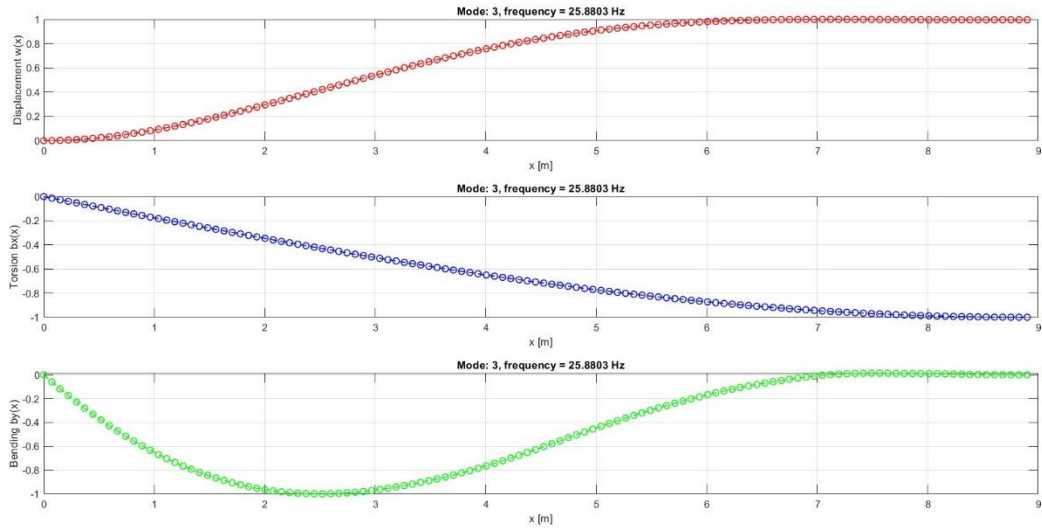


Figure 3. 3rd eigenmode for 25.88Hz/ $w(x)$ -displacement, $b_x(x)$ Torsion, $b_y(x)$ Bending.

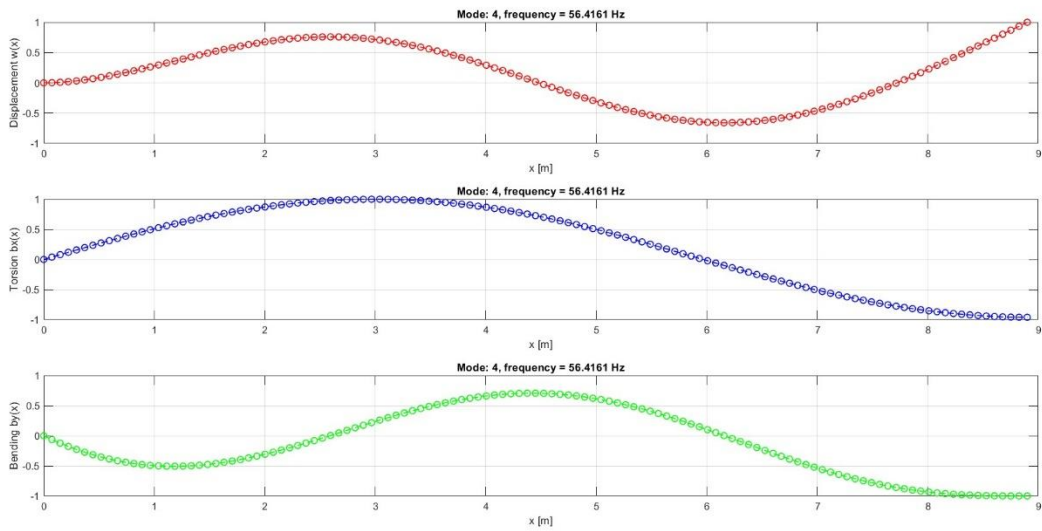


Figure 4. 4th eigenmode for 56.41Hz/ $w(x)$ -displacement, $b_x(x)$ Torsion, $b_y(x)$ Bending.

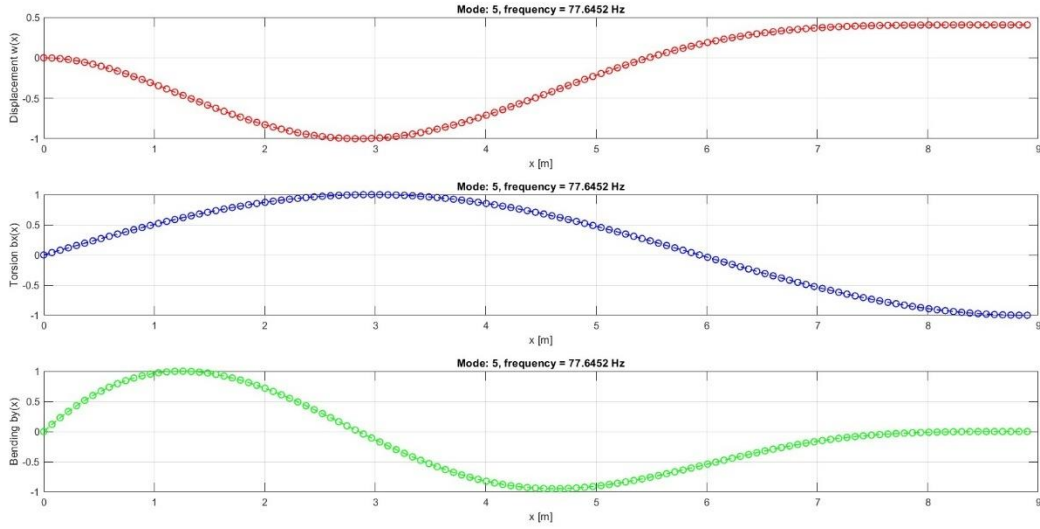


Figure 5. 5th eigenmode for 77.64 Hz/ $w(x)$ -displacement, $b_x(x)$ Torsion, $b_y(x)$ Bending

2.2 Solution in the modal space for an applied force with two different forcing frequencies.

2.2.1 First Part

In the second task, we had to use the first and then the three first eigenvectors of the previous task to calculate the displacement of the last node (wing's tip) by using the superposition method. In order to express the matrices in modal space (generalized), we had to multiply the M, K matrices from the right side with the eigenvectors X (of the previous part) and from the left side with the transposed matrix of the eigenvectors (X') using *equation 15* and *equation 16*. By using the generalized matrices we can solve the reduced equations of motion in modal space, as shown in *equation (25)*.

In this part, we also had to create the vector of the external force by initializing it with the size of the nodes and then setting the force in the middle node (middle of the wing), as stated in the task description. Besides the force vector, we also had to create a time vector. We decided to set a time-step equal to $1/f_{\max}$, where $f_{\max}$ is the maximum natural frequency between the participating modes (first 1 or 3) and the forcing frequency..

The applied force of the first part was equal to:

$$F(t) = 0.1 * \sin(\Omega t) \quad (51)$$

Where the forcing frequency: $\Omega = \frac{\omega_1 + \omega_2}{2}$ (52)

After calculating the Force vector, we had to express it in the modal space, and we achieved that by multiplying it with the transposed matrix of the eigenvectors (X').

For the calculation of time varying generalized displacement vectors (or modal coefficients), that constitute the solutions of the differential equation in the modal space, the process shown in sub-chapter 1.5.1 was followed. We had to calculate the coefficients A, B of the differential equation q_{free} (complementary solution – homogeneous), where:

$$q_{tot} = q_{free} + q_{forced} \quad (53)$$

$$q_{free} = A \cos(\omega_n t) + B \sin(\omega_n t) \quad (54)$$

$$q_{forced} = \frac{F_0}{k} \frac{1}{k - (\frac{\Omega}{\omega_n})^2} \sin(\omega_d t) \quad (55)$$

By applying the initial conditions, we calculated that:

$$A = U(0) \quad (56)$$

$$B = \frac{U'(0)}{\omega_n} - \frac{P_0}{k} \frac{(\frac{\Omega}{\omega_n})}{1 - (\frac{\Omega}{\omega_n})^2} \quad (57)$$

The last step for the calculations in MATLAB was to compute the time varying generalized displacement vectors corresponding to the first three eigenvectors (function: *solve_DE*). In this way, the vectors for the time varying generalized displacement were formed with size - according to the time discretization - of (1x3973), for each one of the three first modes and can be shown in *figure 6, figure 7, figure 8*.

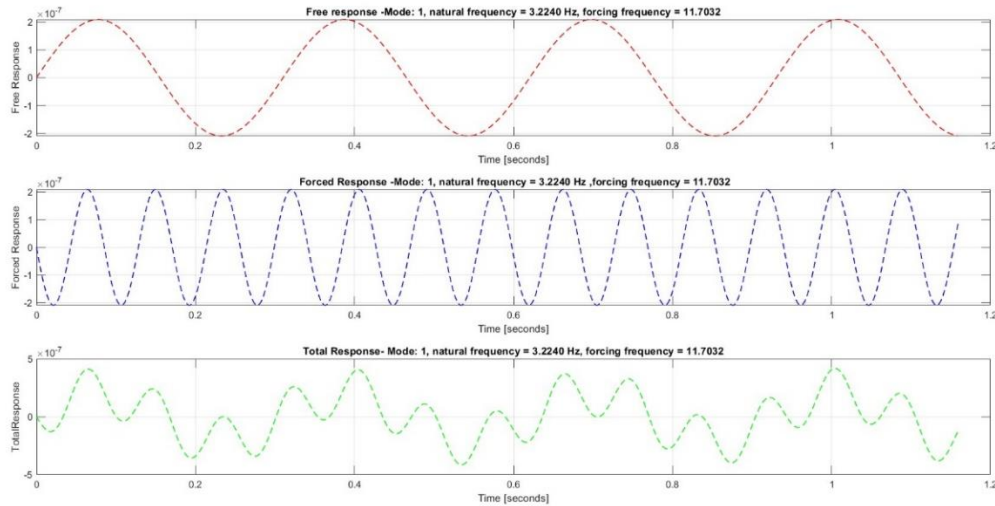


Figure 6. Time varying generalized displacement vector for the first mode.

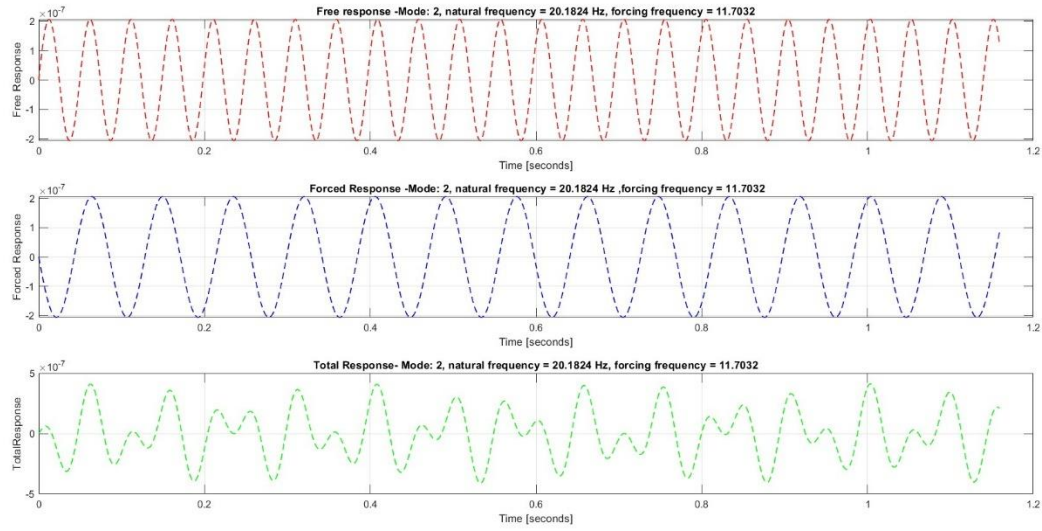


Figure 7. Time varying generalized displacement vector for the second mode.

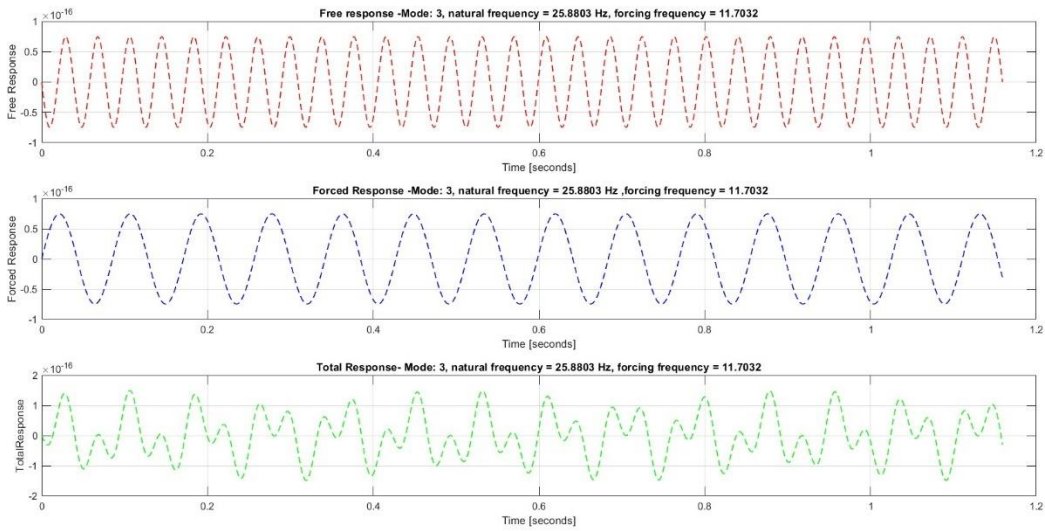


Figure 8. Time varying generalized displacement vector for the third mode.

Finally, the solution in the physical space with the use of only the first eigenvector (1st mode) was calculated by multiplying the first eigenvector with the first time varying generalized displacement vector and can be shown in [figure 9](#). On the other hand, for the case where the first three modes are superimposed, each one of the three eigenvectors was multiplied with its respective time varying generalized displacement vector. Then all

products were added together, as seen in *equation 19*. The physical space solution is presented in *figure 10*.

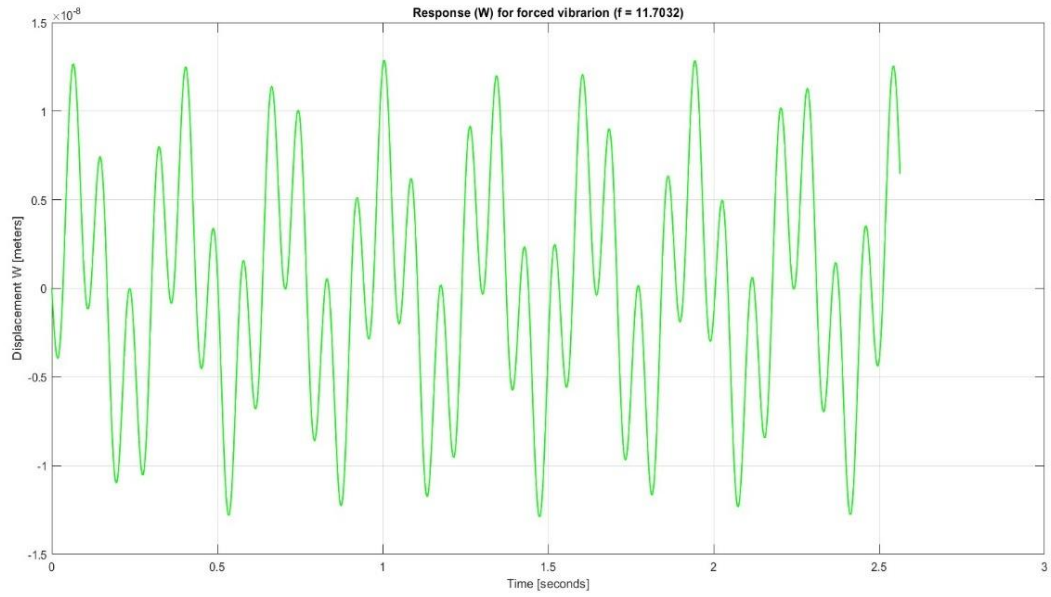


Figure 9. Tip displacement for **one mode**.

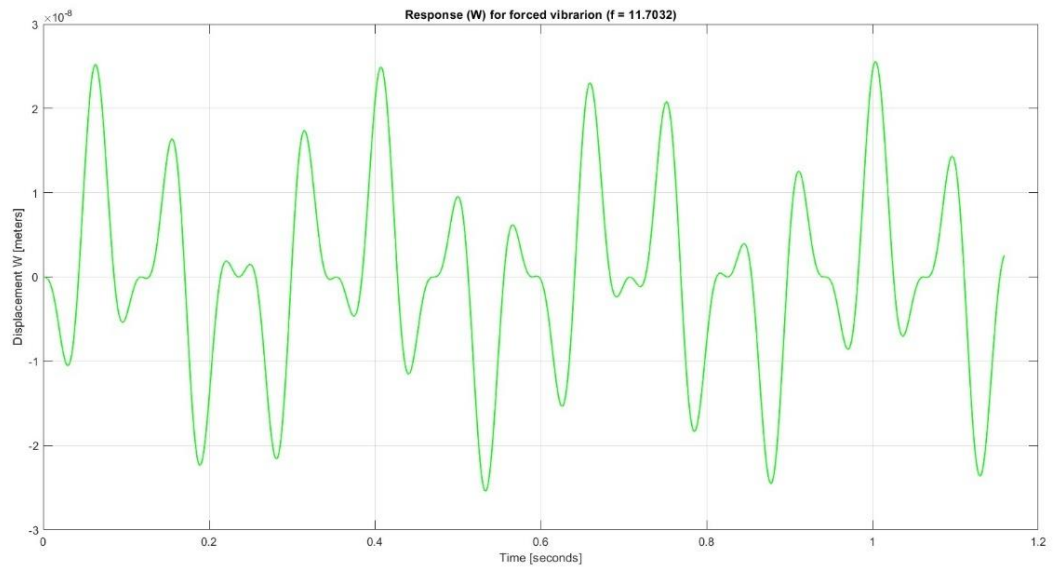


Figure 10. Tip displacement for **three modes**.

2.2.2 Second Part

For the second part of this task, we used the same methodology but replaced the acting force, since it now has different frequency and magnitude. The previous frequency of 11.70 Hz was replaced with the forcing frequency of the engine, and the magnitude was changed from 0.1 to 1. The results derived after this change are shown in *figure 11*.

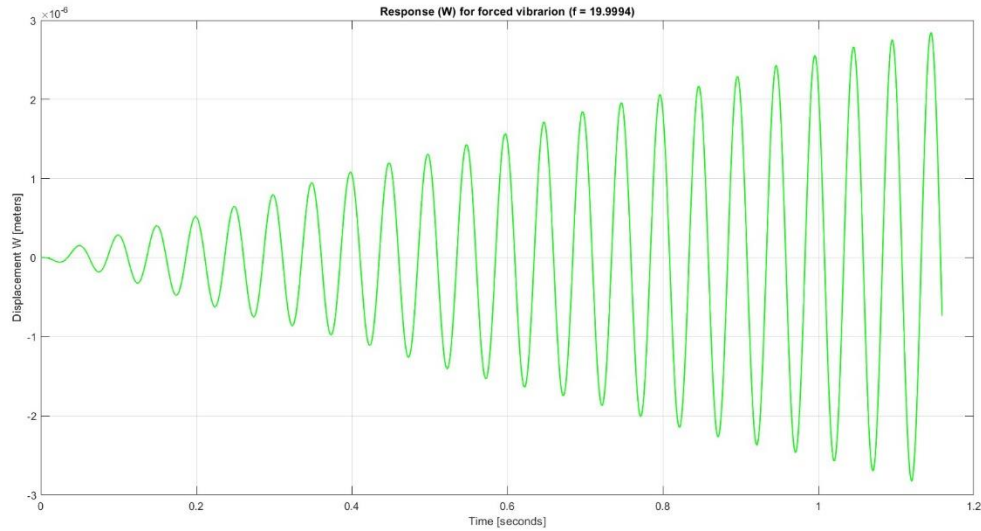


Figure 11. Tip displacement for $f=19.99$ Hz (engine frequency).

Comparing figure 10 and 11, it is obvious that the response in figure 10 differs from the response in figure 11, despite a small change in forcing frequency. In figure 11, the forcing frequency corresponds to 19.99 Hz, which is very close to the 2nd natural frequency of the system. For this reason, the excitation with 19.99 Hz leads to resonance and the response's magnitude continuously increases with time. The response is dominated from the resonance of the 2nd mode. On the other hand, in figure 10, no resonance is displayed since the forcing frequency is in-between the 1st and 2nd natural frequencies. Thus, the response will contain several participating modes, with the 1st and 2nd being excited the most.

2.3 Proportional Reyleigh Damping

The aim of this task is to observe how the dynamic response of the system changes under the same harmonic excitation but with applied damping. The damping is considered to be proportional with the damping matrix equal to $[C]=0.0005[K]$.

Firstly, the damping ratio ζ was calculated with the use of equation (36):

$$2 * \omega_{ni} * \zeta_i = \alpha + b \frac{k_i}{m_i}$$

Where: $\alpha=0$, $b=0.0005$, $k_i=\omega_{ni}^2$, and $m_i=1$.

So: $\zeta_i=\omega_{ni} * 0.00025$ (58)

Because damping is present, the damping frequency is computed with the equation:

$$\omega_{di} = \omega_{ni} \sqrt{1 - \zeta_i^2} \quad (59)$$

After calculating the basic characteristics of the damped system (ζ_i, ω_{di}) , we solve the differential equation according to sub-chapter 1.5.2 .

$$q_{tot} = q_{free} + q_{forced} \quad (60)$$

$$q_{free} = e^{-\zeta_i \omega_{ni} t} [A \cos(\omega_{di} t) + B \sin(\omega_{di} t)] \quad (61)$$

$$q_{forced} = C \cos(\Omega t) + D \sin(\Omega t) \quad (62)$$

The coefficients A,B,C,D were calculated by applying the initial conditions.

$$C = \frac{P_0}{\omega_n^2} \frac{1 - (\frac{\Omega}{\omega_n})^2}{(1 - (\frac{\Omega}{\omega_n})^2)^2 + (2\zeta_i \frac{\Omega}{\omega_n})^2} \quad (63)$$

$$D = \frac{P_0}{\omega_n^2} \frac{-2\zeta_i \frac{\Omega}{\omega_n}}{(1 - (\frac{\Omega}{\omega_n})^2)^2 + (2\zeta_i \frac{\Omega}{\omega_n})^2} \quad (64)$$

$$A = -D \quad (65)$$

$$B = \frac{\zeta_i A \omega_n - \Omega C}{\omega_{di}} \quad (66)$$

For the next steps, the methodology of the previous task is followed, and the method of mode superposition is implemented.

The displacement in the physical space for only the first eigenvector was calculated by multiplying the first eigenvector with the first time varying generalized displacement vector and is presented in [figures 12 and 13](#) for a forcing frequency of 11.70 Hz and 19,99 Hz, respectively. For the case where three modes are used, each of the three eigenvectors is multiplied by the corresponding time varying generalized displacement vector and then their products are super positioned, according to [equation 19](#). The displacement is shown in [figure 14](#) for forcing frequency 11.70 Hz and in [figure 15](#) for forcing frequency 19,99 Hz.

Finally, in *figure 16* we present a plot for the comparison of the undamped system's response and the damped system's response, with the use of three modes observed.

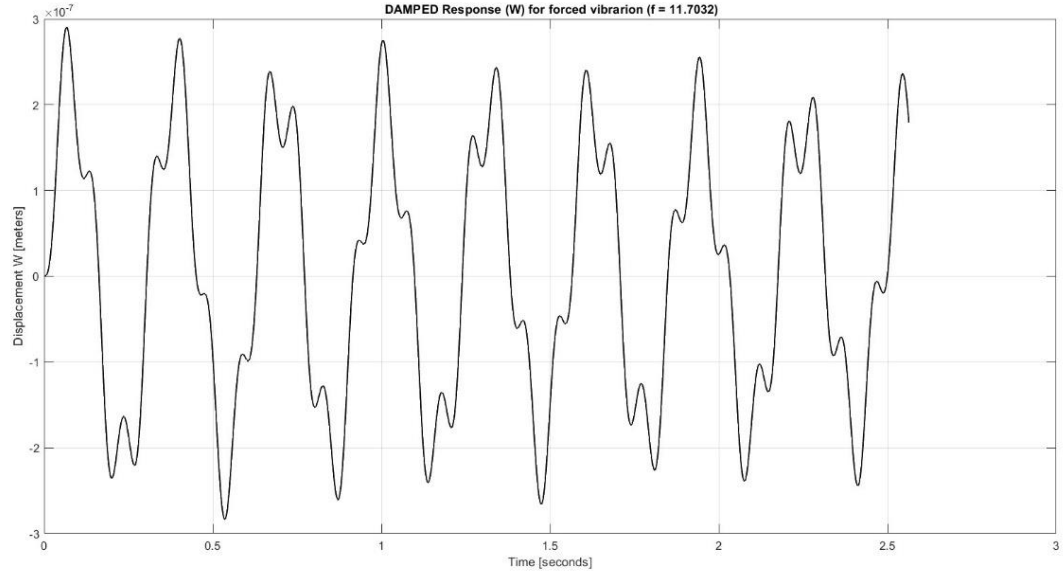


Figure 12. Tip displacement for one mode with damping at 11.7 Hz.

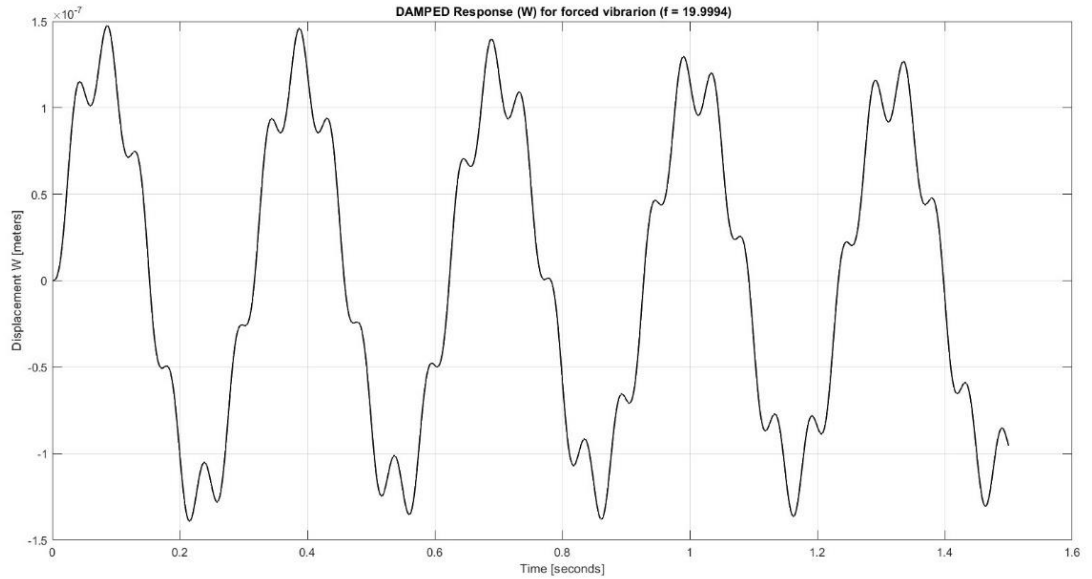


Figure 13. Tip displacement for one mode with damping at 19.99 Hz.

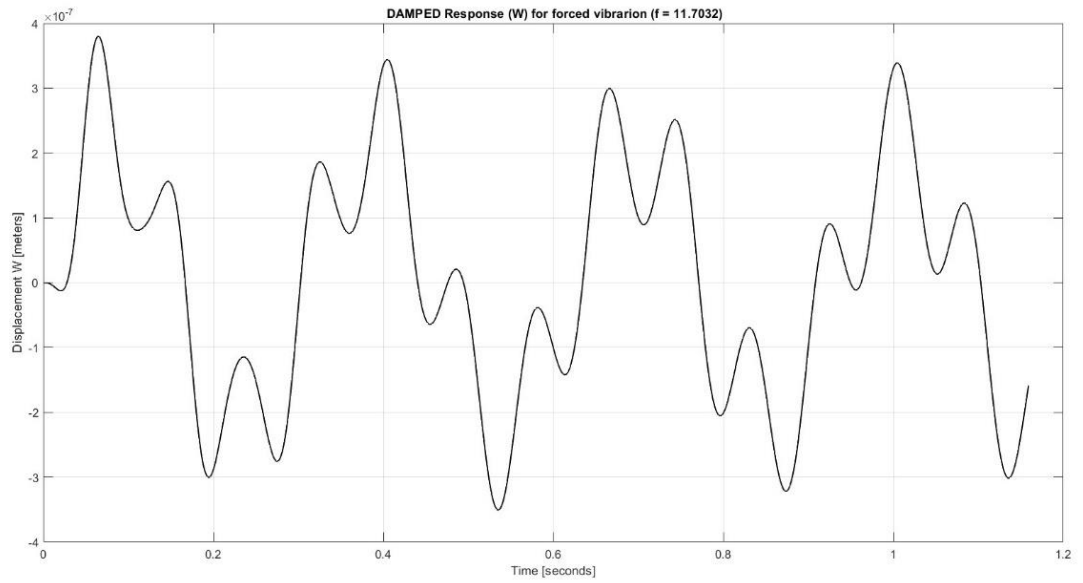


Figure 14. Tip displacement for three modes with damping at 11.70Hz.

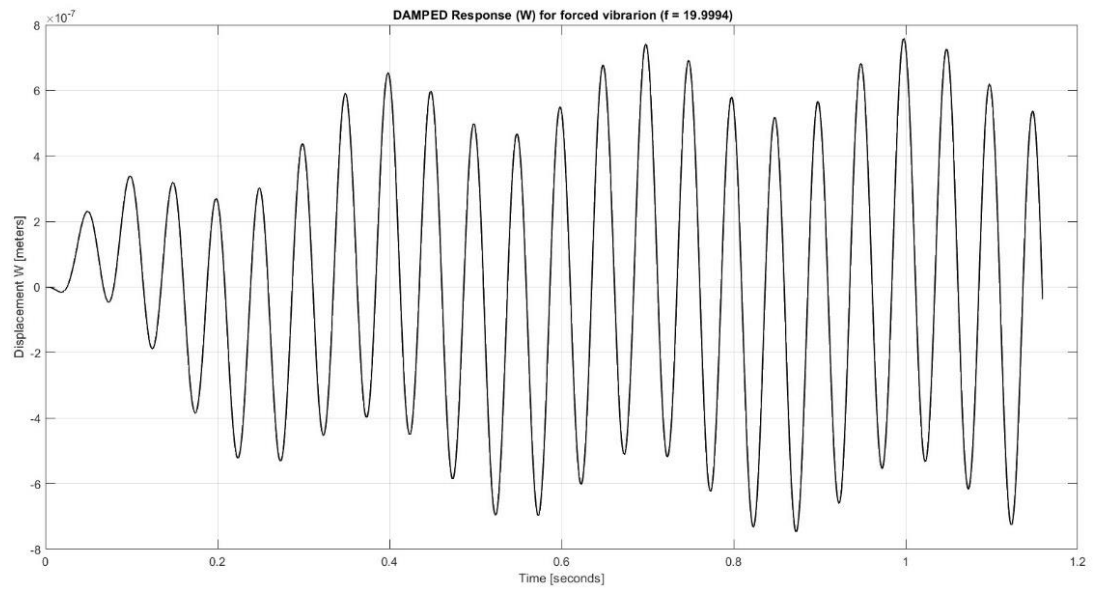


Figure 15. Tip displacement of the tip for three modes with damping at 19.99Hz.

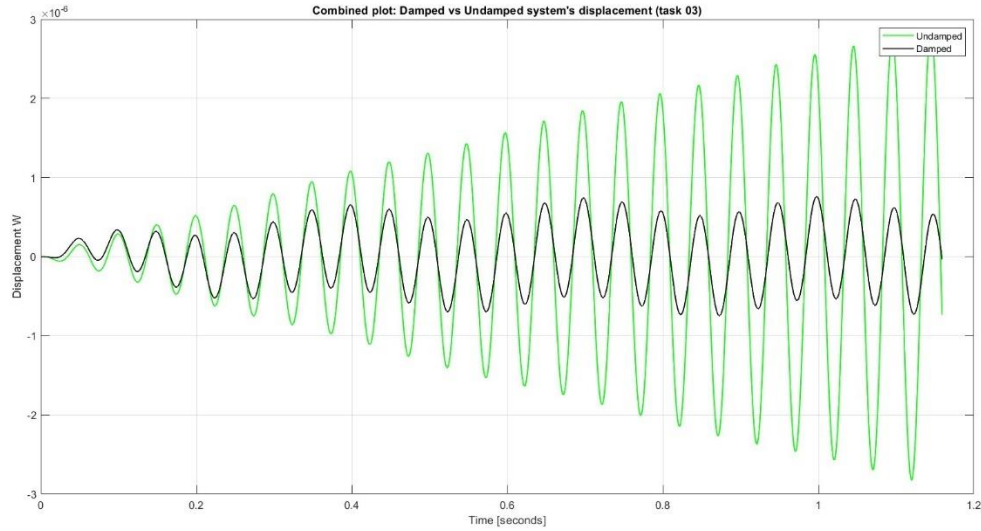


Figure 16. Damped vs Undamped for three modes at 19.99 Hz.

Figure 16 presents the time history of the transverse displacement at the tip of the wing for both the undamped and the proportionally damped system, under the same harmonic excitation 19.99 Hz (engine).

In the undamped case, the response exhibits a continuously increasing oscillation amplitude over time. This behavior indicates sustained energy accumulation due to the harmonic excitation acting close to the system's natural frequencies, resulting in a resonant-type response. Since no dissipation mechanism is present the vibration energy is not attenuated leading to progressively larger displacements. In contrast, the damped response shows a significantly reduced amplitude throughout the simulation. Although oscillatory motion is still present, the damping mechanism effectively limits the vibration growth by dissipating part of the input energy. As a result, the system reaches a bounded steady-state response, preventing the excessive displacement levels observed in the undamped configuration. Moreover, a slight phase shift between the damped and undamped responses can be observed, which is a characteristic of viscously damped systems. The overall comparison clearly demonstrates the stabilizing effect of proportional damping on the dynamic behavior of the wing.

In conclusion, the introduction of a small stiffness-proportional damping coefficient ($[C]=0.005[K]$) is sufficient to significantly reduce vibration amplitudes and ensure a physically realistic response, emphasizing the critical role of damping in the dynamic performance and structural integrity of structures and specifically aircraft wings.

2.4 Response of damped system via the complex eigensolution

In this task, the damping of the system will be treated differently. In general, the damping matrix may not be symmetric and for this reason the standard eigenvalue problem is formulated and solved differently via the complex eigensolution. The system of 2nd-order differential equation of $n \times n$ size (n are the degrees of freedom) will be re-expressed in state-space form as a system of 1st order differential equations of size of $2n \times 2n$. Since the system's size is $2n \times 2n$, there are of $2n$ eigenvalues and of $2n$ eigenvectors and for the proper handling of the damping, left and right eigenvectors need to be computed. The methodology used is presented with detail in chapter 1.6.

Before computing the right and left eigenvectors, the new system's matrices need to be found. Matrix A is computed based on equation (38). It is important to note that there is no forcing factor, so the second term of the right-hand side (the term B) is ignored. The system's eigenvalues and eigenvectors (right) can be found with the use of the MATLAB's command *eig*. The left eigenvectors are computed based on the adjoint system so the adjoint matrix of A needs to be used. There is no need to compute the left eigenvalues since they are identical to the right eigenvalues. The eigenvalues, as well as both left and right eigenvectors, are in complex form since they include an imaginary part. At this point, the response of the system as a function of time can be solved and the only requirement is to define the initial conditions that govern the problem. If initial conditions are set to zero, then the system has no response, which can be evaluated from *equation (41)*. Since no initial conditions were given in the assignment, an exponential expression as a function of node position in the wing was defined (*equation 67*) with size of $2n \times 1$, out of which the displacement of the latter n elements were set to zero, with the goal of simulating an initial bending shape.

$$\text{Initial conditions} = 10^{-4} \cdot e^x \quad (67)$$

Before continuing with the solution, it is ensured that the right and left eigenvectors comply with the biorthonormality condition, as mentioned in sub-chapter 1.6. The response was computed based on *equation (41)* and the wing's tip displacement is presented in *figure (17)*:

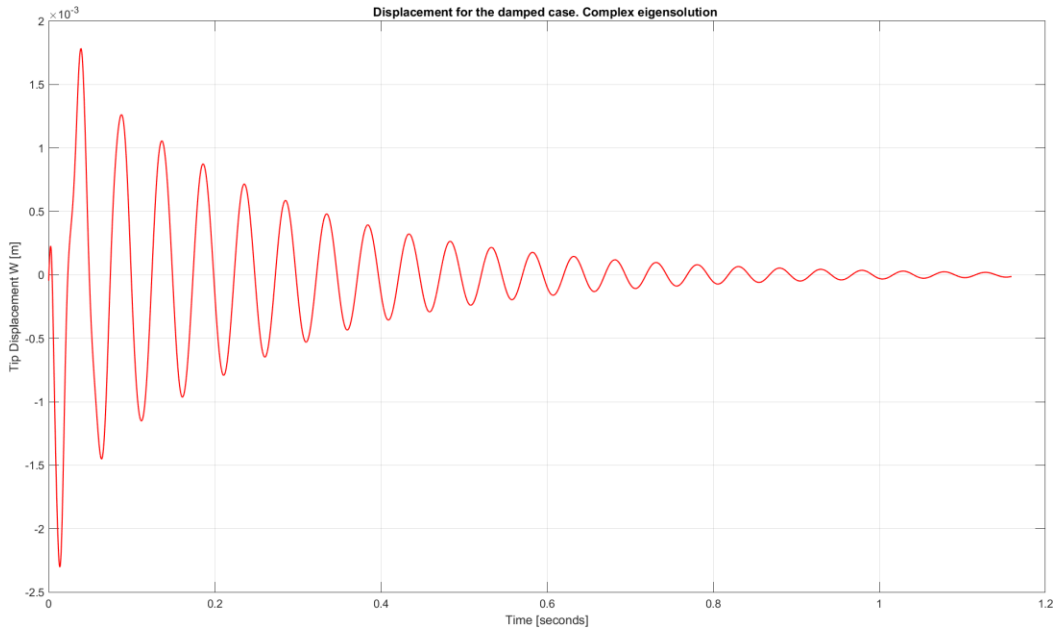


Figure 17: Free response of the wing's tip displacement for the damped system.

It is evident that the damping of the system is present since the response decays with time. The MATLAB implementation can be found in the appendix.

2.5 Transient response with the use of explicit numerical integration

The response of the wing as a function of time can be computed, either by the analysis in the modal space and superposition of eigenmodes or by using numerical integration schemes. In this task, the explicit numerical integration method of central differences is going to be implemented in order to find and present the transient response of the tip displacement as a function of time. The scheme and the steps that are needed have been presented in sub-chapter 1.7.1. Since the system has no initial conditions (initial displacement and velocity are zero) the initial acceleration at $t=0$ is also zero. The system experiences a sinusoidal time varying load forcing in the middle node, whose frequency (as shown in task 2) is equal to $f = 11.7 \text{ Hz}$. The numerical scheme is implemented with a for loop where the values of the displacement are updated on each passage, where each passage corresponds to the system's response for a moment in time. Numerical integration necessitates a time step in order to discretize a specified time period. The time step will be set twice as $dt1 = \frac{T1}{5}$ and $dt2 = \frac{Tmax}{5}$, where $T1$ and $Tmax$ are the periods that correspond to the minimum and maximum computed natural frequencies (the first and last),

respectively. Table (2) includes the main properties that were used, in order to define the time steps.

Numerical Integration			
	Period [sec]	Natural Frequency [Hz]	Time step [sec]
Case 1	$T1 = 0.310 \text{ sec}$	$f1 = 3.224 \text{ Hz}$	$dt1 = 0.062$
Case 2	$Tmax = 1.276 \cdot 10^{-5}$	$fmax = 7.835 \cdot 10^4$	$dt2 = 2.556 \cdot 10^{-5}$

Table 2: Numerical Integration properties

Before proceeding with the solution, the total time period for which the response will be computed must be defined. Since the time step has been defined from the lowest and largest natural frequency, the time period will be set in a similar manner so as to compute the solution for a sufficient time window. For the first case, the time period is set as 1000 periods (corresponding to the first natural frequency) and for the second case it is set as 10^4 periods (corresponding to the last natural frequency). The 2nd case requires more periods to provide a satisfactory solution since Tmax is smaller than T1 for about four orders of magnitude. It is understood that the second case will have increased computational time due to the very small time step and the amount of time steps needed to compute the response for a reasonable time period. As the finite element discretization increases, the last natural frequency increases, thus computational cost rises substantially. This remark is the main disadvantage of the central differences method. The results for time steps dt1 and dt2 are presented in figures... From chapter 1, it is known that the largest natural frequency dictates the time step and that the time step should always be smaller than $\frac{T_{max}}{\pi}$ (equation 44) in order to avoid numerical instability and failure. As a result, a failed response for case 1 is expected whereas case 2 is expected to be successful.

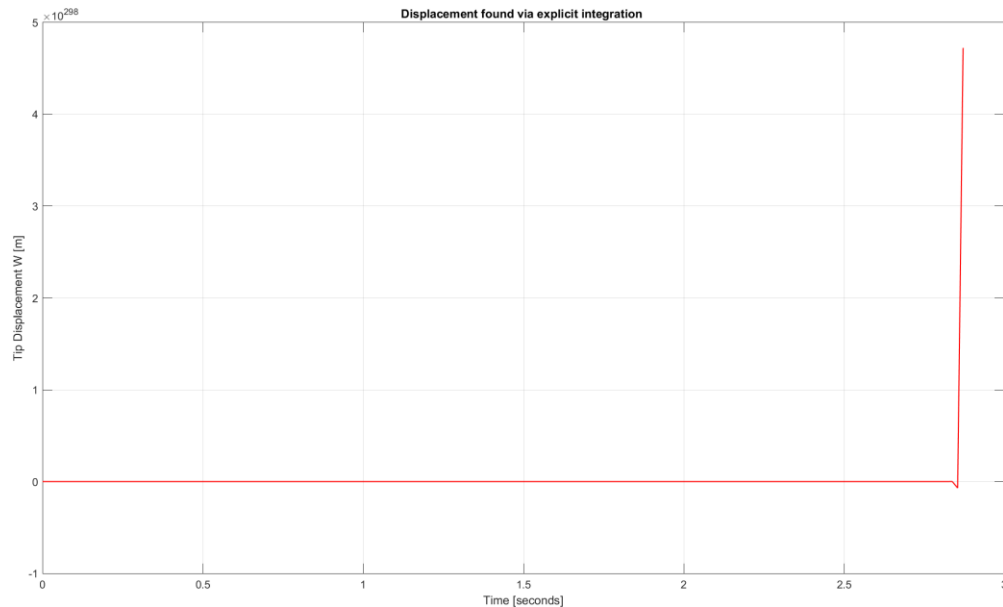


Figure 18: Tip displacement for $dt1=T1/5$.

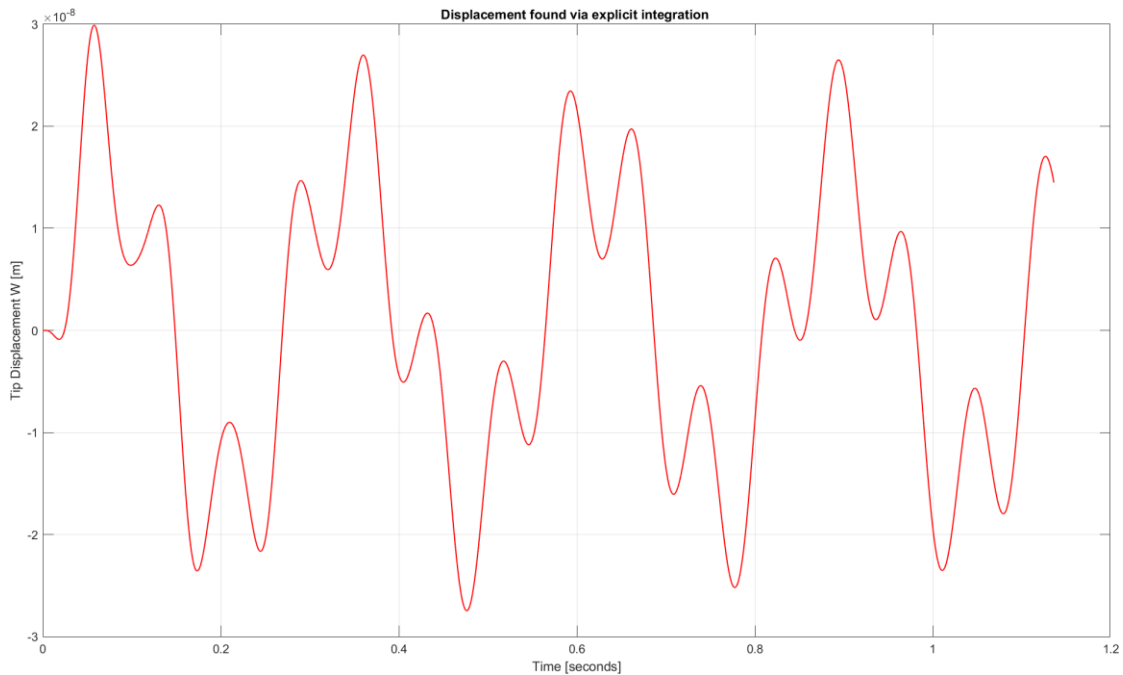


Figure 19: Tip displacement for $dt2=T_{max}/5$.

Indeed, as seen in figure (18) and (19), the use of $dt1$ has led to the failure of the numerical scheme and the response hasn't been computed. On the other hand, the use of $dt2$ did not cause any numerical instability and the response of the damped system has been computed successfully. The derived response can be compared with the response computed from the proportional damping approach (task 3) so as to evaluate the results. From figures (12) and (14) we conclude that both responses are similar, thus proving that both methodologies are correct. However, a remark can be made regarding the efficiency of each methodology. The approach of proportional damping and modal space usage has far less computational costs and overall lower run-time than the numerical integration approach and, at the same time, provides an almost identical solution. On the other hand, the explicit numerical integration involves a costly for-loop due to the very small time steps. As a result, it can be understood that numerical integration is not the most effective approach.

2.6 Transient response with the use of explicit numerical integration and generalized/modal matrices

In the previous task, the computational costs of numerical integration were evident. A method that can be implemented to reduce run-time is the usage of modal/generalized

matrices instead of the physical space matrices. The generalized matrices are diagonal, thus allowing for easier handling and faster calculations. It must be noted that the solution of the numerical integration is in modal coordinates and in order to have physical meaning, it must be multiplied with the modal matrix (equation 23). In figure(20) , the response is shown for the 1st case, where the time step is $dt1$.

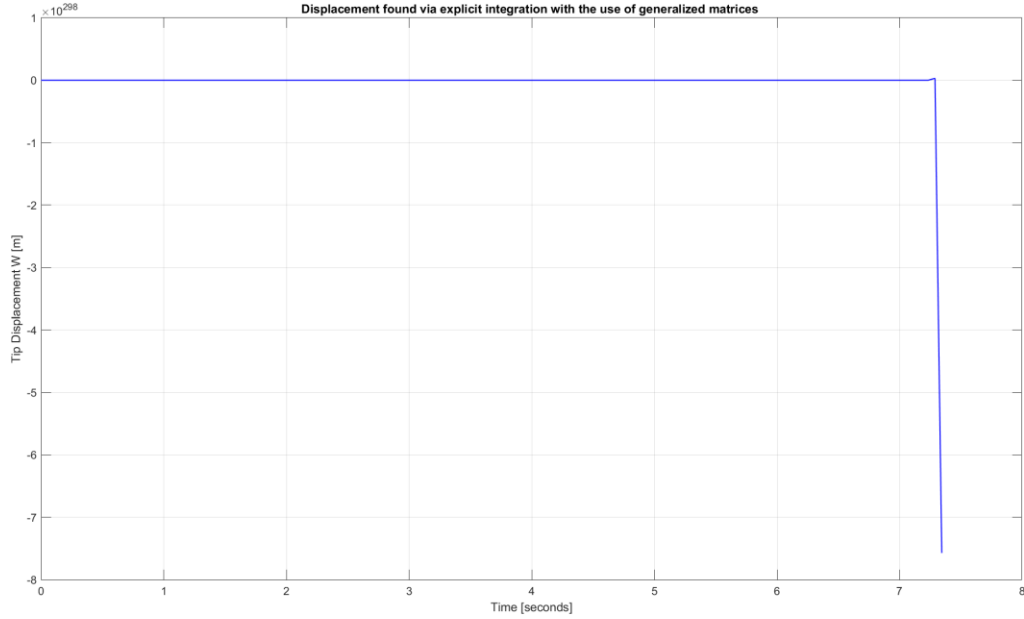


Figure 20: Response with time step $dt1=T1/5$ and modal matrices

The use of $dt1$ as time step breaks the numerical scheme and similarly to the previous task, no solution is derived. For the 2nd case where the time step is $dt2$, the response is shown in figure (21) .

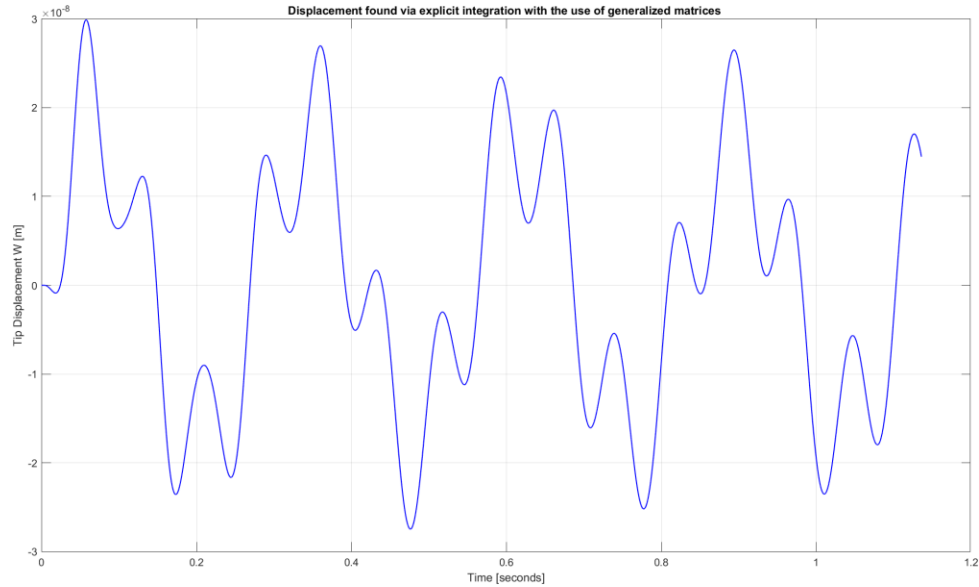


Figure 21: Tip displacement for $dt2 = Tmax/5$ and modal matrices.

The two responses (with physical matrices and modal matrices) are compared in figure (22) .

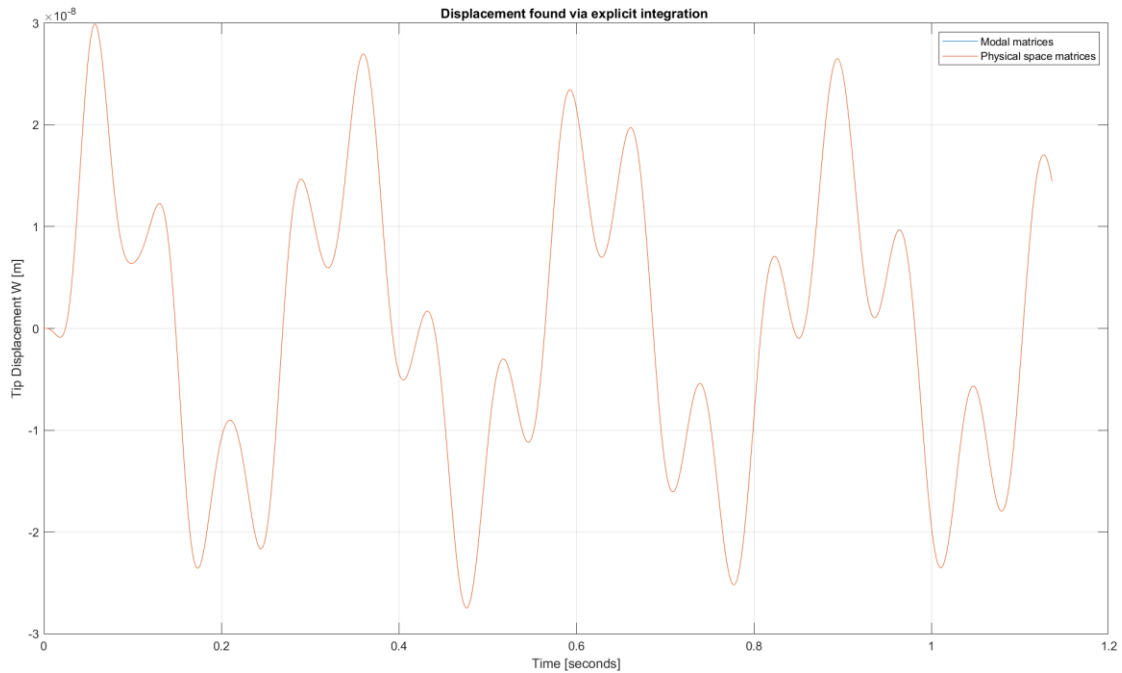


Figure 22: Comparison of tip response via numerical integration with physical space and modal space matrices

The figures indicate that there is no difference between the two responses and thus identical results are derived from both methodologies. In essence, the numerical integration implementation matches the results of previous tasks where modal analysis was used, thus validating the methodology that was followed . The physical system has been sufficiently expressed in the modal space, and the usage of modal matrices yields equivalent solutions to those derived in physical space.

2.7 Aeroelasticity

The last part of this assignment focuses on the aeroelasticity of the wing and aims to simulate the dynamic response of the wing when uniform air flow flows over it. Figure 23 makes it clear that the closer to the tip of the wing, the tip rotation is greater, which means that the aerodynamic loads are calculated for different angles in each section (airfoil), thus producing loads with different magnitudes.

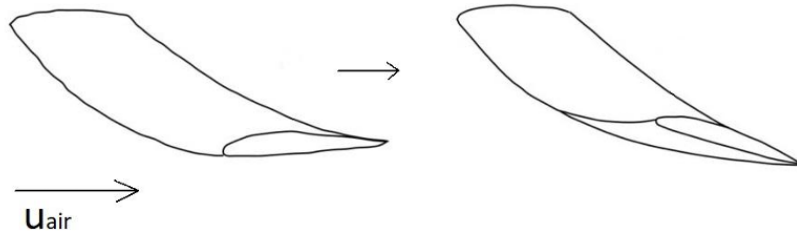


Figure 23. Indicative sketch of a wing subjected to aeroelastic loading.

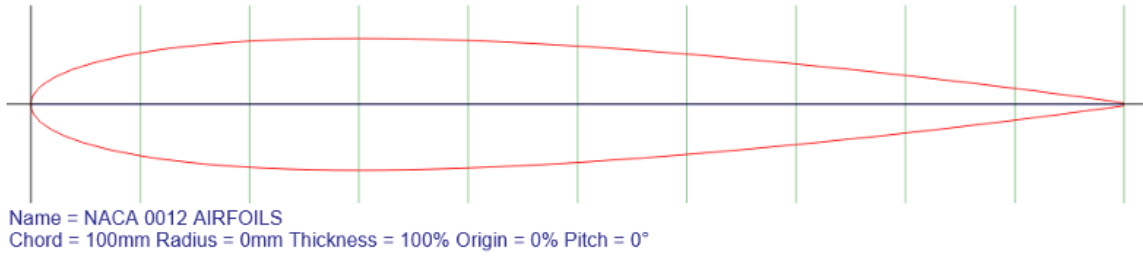


Figure 24. Airfoil geometry and characteristics of the NACA 0012 airfoil (airfoiltools.com).

For the first part of this task, we consider the wing as a solid box (equivalent behavior with the airfoil NACA 0012) as we did on the previous tasks. We integrated a given MATLAB function (Air_load.m) that calculates the aerodynamic loads (lift, drag) the and pitching moment that acted on each of the blade's/wing's node. This function takes as inputs: air velocity (m/sec), chord length (m), initial angle of attach (rad), the solution of the previous step of the Newmark analysis which is a vector with all the displacements (w, bx and by), the number of degrees of freedom (DoFs), the surface of each element and the number of elements, and gives as output the vector R which contains all the applied forces of the next time step.

To initialize the force, we set the angle at 1° degree. After that we had to calculate the acceleration for $t=0$ in order to calculate the first value of the displacement $U(t)$ with the Newmark methodology (sub-chapter 1.7.2). The equation for the calculation of the acceleration is *equation (70)*.

External load:

$$R(0) = \text{Air_load}(20, 2, \text{rad2deg}(1^\circ), U(0), 3, S_{el}, n)$$

Initial acceleration:

$$\ddot{U}(0) = M^{-1} [R(0) - C \cdot \dot{U}(0) - K \cdot U(0)] \quad (68)$$

The solution based on the Newmark Method can be calculated with the below equations:

Time-stepping equations:

Displacement:

$$U_{i+1} = (h_1 M + b h_2 C + K)^{-1} [R_{i+1} + M(h_1 U_i + h_2 \dot{U}_i + (1/(2a) - 1)U_i) + C(b h_2 U_i + (b/a - 1)U_i + (\Delta t/2)(b/a - 2)\dot{U}_i)] \quad (69)$$

Acceleration:

$$\ddot{U}_{i+1} = (1/(a\Delta t^2))(U_{i+1} - U_i) - (1/(a\Delta t))\dot{U}_i - (1/(2a) - 1)\ddot{U}_i \quad (70)$$

Velocity:

$$\dot{U}_{i+1} = \dot{U}_i + (1 - b)\Delta t \cdot \ddot{U}_i + b\Delta t \cdot \ddot{U}_{i+1} \quad (71)$$

Where h_1, h_2 are auxiliary constants:

$$h_1 = \frac{1}{a \cdot \Delta t^2} \quad (72)$$

$$h_2 = \frac{1}{a \cdot \Delta t} \quad (73)$$

When the for loop is completed, we plot the tip displacement (w) and the tip rotation(bx) as figure 25 shows.

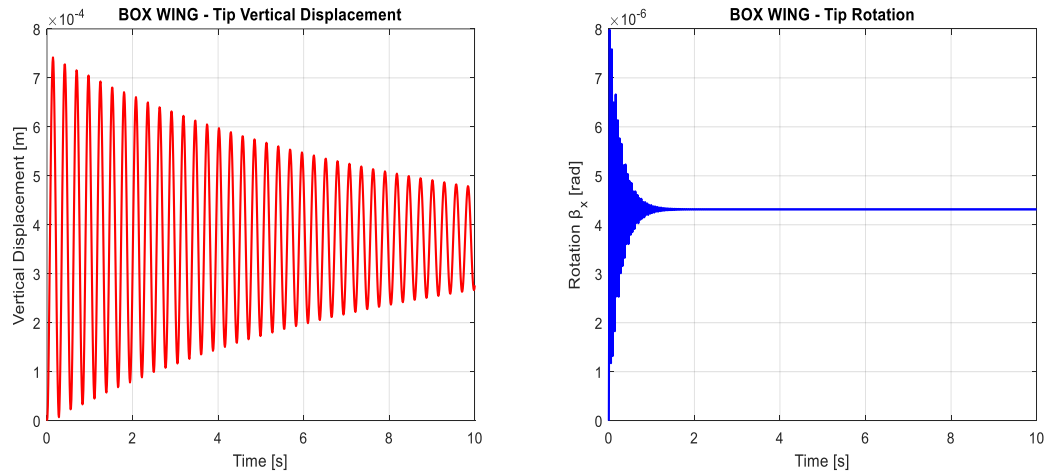


Figure 25. Tip Vertical displacement and tip rotation for BOX WING (20 m/s).

For the second part of this task, we had to consider the wing structure as an I beam instead of a solid box as figures 26 and 27 illustrate.



Figure 26. Realistic profile of the wing.

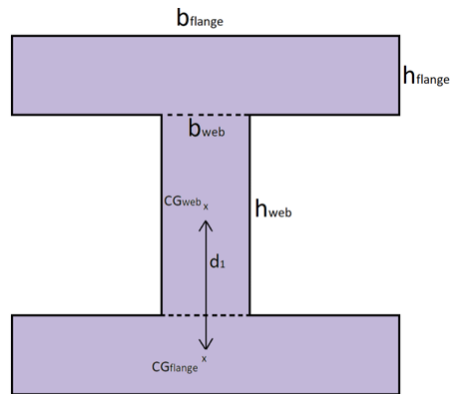


Figure 27. Profile beam type I.

For the second case, it was assumed that the stiffness of the wing was concentrated in the I-beam section. Specifically, the following assumption was made: the aerodynamic loads were calculated based on the airfoil, while the structural component that carries these loads was solely the I-beam, as illustrated in figures 26 and 27. Due to that, we had to recalculate the stiffness, mass and damping matrices since the moments of inertia I_z, I_y J_p of the I beam are different from the box wing. Hence, we selected the beam section characteristics as the table below describes (table 3).

Table 3. I-Beam characteristics.

Flange width	0.2 m
Flange height	0.0225m
Web width	0.02m
Web height	0.105m

The following resolution process does not differ from the previous Newmark methodology. The plot that arose is illustrated in figure 28.

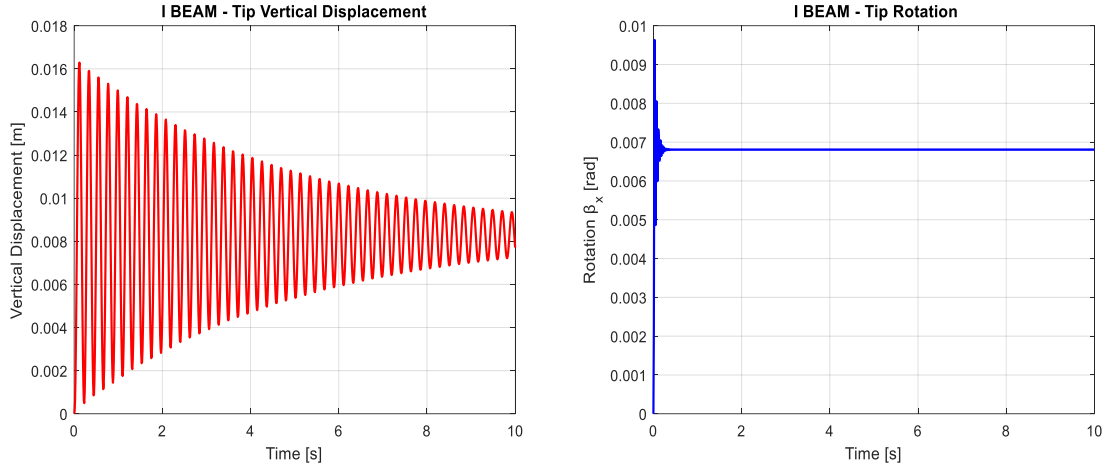


Figure 28. Tip Vertical displacement and tip rotation for I BEAM WING (20 m/s).

Figures 25 and 28 compare the time histories of the tip vertical displacement and tip rotation for the wing whose structure is modelled as a box section and as an equivalent I-beam under identical aeroelastic loading conditions. The results clearly demonstrate the influence of the structural stiffness distribution on the aeroelastic response.

For the box wing configuration, the vertical displacement exhibits oscillatory behavior with a gradually decaying amplitude, indicating a stable response dominated by bending vibrations. The displacement levels remain relatively low due to the high bending stiffness of the closed box section. The corresponding tip rotation decays rapidly during the transient phase and converges to a very small steady value, reflecting the high torsional rigidity of the box wing and its ability to effectively suppress torsional deformation. In contrast, the I-beam model shows significantly larger vertical displacement amplitudes, indicating reduced bending stiffness. More importantly, the tip rotation response is substantially higher before reaching steady state, highlighting the much lower torsional stiffness of the open I-beam section. This behavior reveals stronger aeroelastic coupling and increased sensitivity to torsional effects. Overall, while both configurations remain aero elastically stable for the examined conditions ($V = 20$ m/s), the I-beam model is considerably more flexible, particularly in torsion. The box wing provides superior structural rigidity and results in reduced displacements and rotations, leading to a more realistic and robust aeroelastic response. However, the box -wing is obviously an unrealistic case and by no means a wing is designed in this way.

The above remarks were also evaluated when different air velocities were tested. The figures that were previously presented, corresponded to air velocity of 20 m/s. When the air velocity is increased to 70 m/s, the wing whose internal structure is assumed as an I-beam displays static divergence while the box- wing does not. Even if the velocity is increased to levels above Mach 1, the box wing still does not display any static divergence. It is understood that the box wing is extremely stiff, meaning that the velocity at which

static aeroelastic divergence is observed is substantially large. On the other hand, the I-beam is considerably less stiff which ,in turn, results in the appearance of static divergence in low velocities. These observations are mathematically true and can be derived from equation (74)

$$V_{divergence} = \sqrt{\frac{2K}{\rho S C L_a e}} \quad (74)$$

Where K is the stiffness, ρ the air density , S the surface , $C L_a$ the slope of lift coefficient with regards to angle of attack, and e the distance between the aerodynamic center and the center of rotation. The larger the stiffness, the larger the velocity at which static divergence occurs.

In figures 29 and 30 , the tip displacement and rotation are presented for air velocity of 70 m/s.

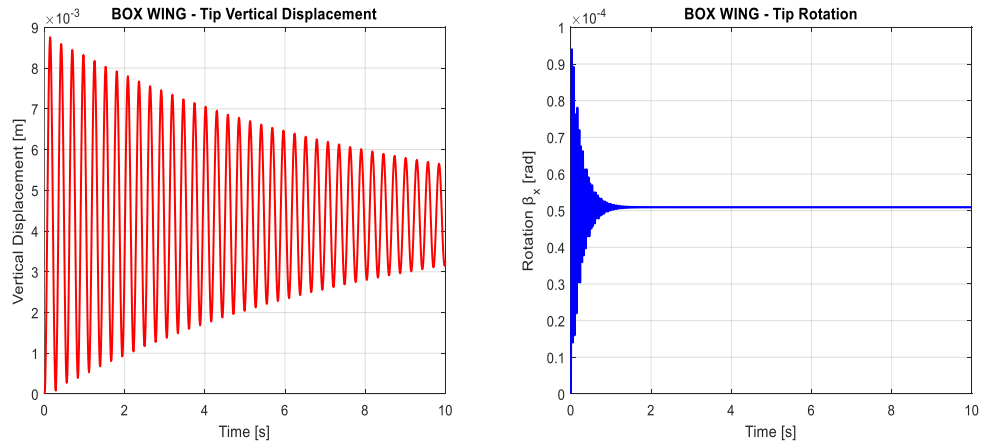


Figure 29: Tip Vertical displacement and tip rotation for BOX WING (40 m/s).

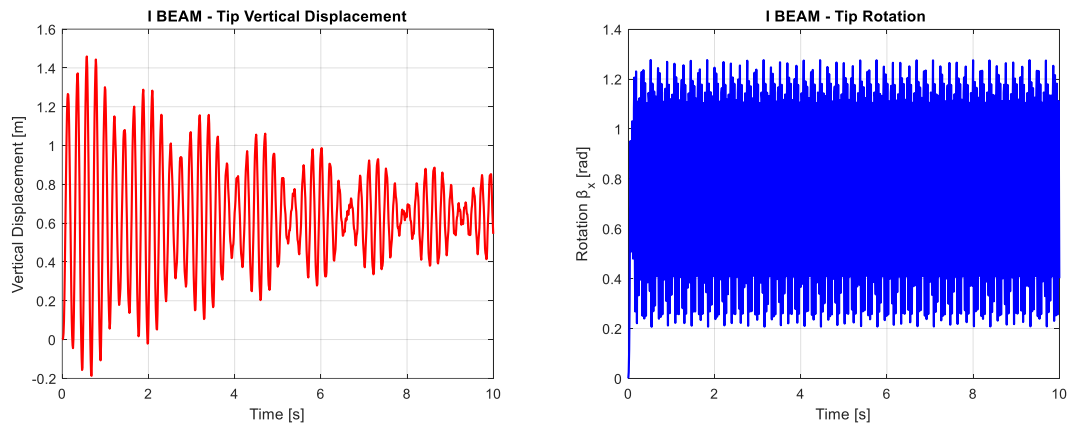


Figure 30: Tip Vertical displacement and tip rotation for I-BEAM(40 m/s).

3 Bibliography

- [1] Review of Dynamics of Structures: Theory and Applications to Earthquake Engineering, Third Edition, by Anil K. Chopra.
- [2] Hoen, Christopher, and Vetco Aibel As. "An engineering interpretation of the complex eigensolution of linear dynamic systems." *Proceedings of International Modal Analysis Conference XXIII*. SEM, Society of Experimental Mechanics, 2005.
- [3] Rao, Singiresu S., and Fook Fah Yap. *Mechanical vibrations*. Vol. 4. New York: Addison-Wesley, 1995.

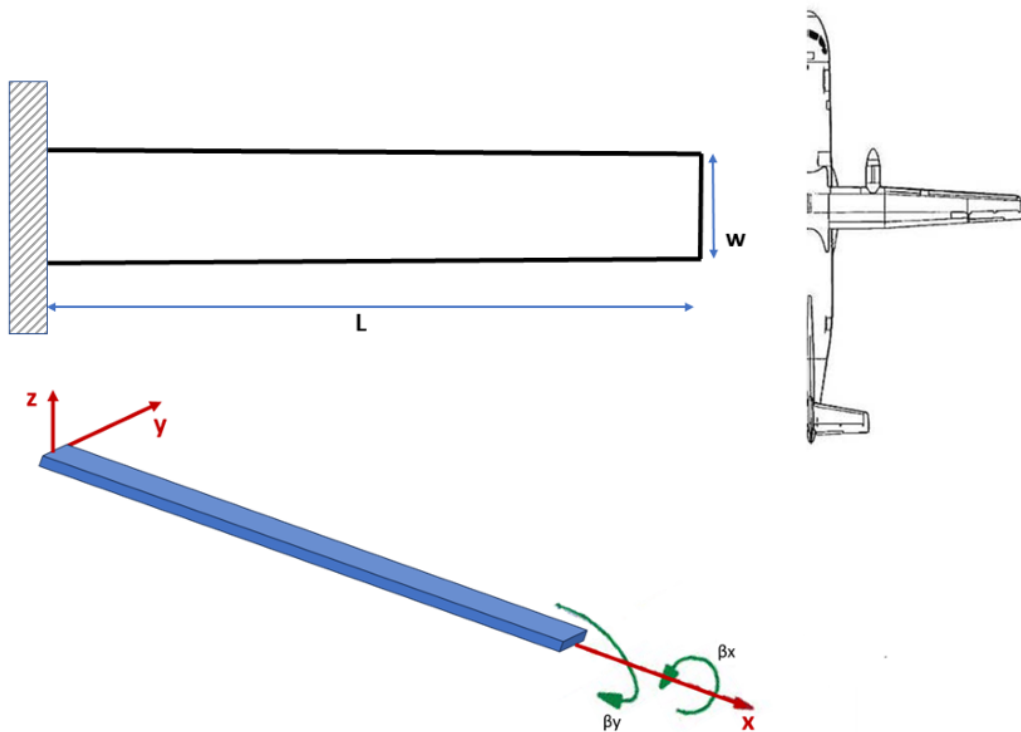
4 Appendix

ΔΥΝΑΜΙΚΗ ΚΑΤΑΣΚΕΥΩΝ 2025-2026

ΜΕΡΟΣ 1^ο

1^ο ΘΕΜΑ

Θεωρείστε τη πτέρυγα του αεροσκάφους της Εικόνα 1 η οποία θεωρείται ότι βρίσκεται πακτωμένη στην άτρακτο του αεροσκάφους. Η πτέρυγα είναι κατασκευασμένη από Carbon/Epoxy, και έχει σταθερό πάχος και πλάτος σε όλο της το μήκος. Η συγκεκριμένη πτέρυγα να προσομοιωθεί με ένα πλέγμα στοιχείων δοκού καθένα από τα οποία θα έχει το ίδιο μήκος. Καθένα από τα στοιχεία δοκού θα χρησιμοποιεί κινηματικές υποθέσεις δοκού Timoshenko με κομβικούς βαθμούς ελευθερίας $\{w, b_x, b_y\}$.



Εικόνα 1 Πτέρυγα αεροσκάφους

Για την πτέρυγα δίνονται τα ακόλουθα χαρακτηριστικά (Δεν είναι οριστικά, θα υπάρξει ενημέρωση):

Blade Thickness $h=0.15\text{m}$

Blade Length $L=8.9\text{m}$

Blade width $W=2\text{m}$

Elasticity Modulus $E=160\text{ GPa}$

Poisson Ratio $\nu=0.275$

Density $\rho=1578 \text{ kg/m}^3$

Ζητείται στην πτέρυγα να πραγματοποιηθούν οι ακόλουθοι υπολογισμοί για την δυναμική της συμπεριφορά αναπτύσσοντας ένα ευέλικτο κώδικα σε περιβάλλον MATLAB. Τα χαρακτηριστικά της πτέρυγας θα πρέπει να δίνονται στην αρχή του κώδικα ενώ οι προσομοίωση της πτέρυγας θα επιτρέπει μεταβαλλόμενο αριθμό στοιχείων. Η επίλυση να γίνει με χρήση συνεπούς μάζας.

Ζητούνται:

1. Να υπολογισθούν και να παρουσιαστούν οι φυσικές συχνότητες του συστήματος και οι 5 πρώτες ιδιομορφές. Στο σημείο αυτό η προσομοίωση θα πρέπει να εξασφαλίσει ότι το πλέγμα πεπερασμένων στοιχείων είναι επαρκές για να επιτευχθεί σύγκλιση για τις 10 πρώτες ιδιοσυχνότητες.
2. Να κατασκευασθεί το ισοδύναμο δυναμικό σύστημα χρησιμοποιώντας την μέθοδο υπέρθεσης ιδιομορφών χρησιμοποιώντας την πρώτη και κατόπιν τις τρεις πρώτες ιδιομορφές. Να υπολογισθεί στο ελεύθερο άκρο της πτέρυγας:
 - a. Η εξαναγκασμένη ταλάντωση με αρμονική διέγερση δύναμης $F(t)=0.1\sin(\omega t)$ N στο μέσο $(L/2)$ της πτέρυγας, όπου ω θα είναι ο μέσος όρος των 2 πρώτων φυσικών συχνοτήτων.
 - b. Η εξαναγκασμένη ταλάντωση με αρμονική διέγερση δύναμης $F(t)=\sin(\omega_{\text{engine}}t)$ N στο μέσο $(L/2)$ της πτέρυγας, όπου ω_{engine} είναι η συχνότητα λειτουργίας του κινητήρα. Δίνεται στροφές λειτουργίας κινητήρα 1200 RPM.

Θεωρείστε κατόπιν ότι υπάρχει βισκώδης δυναμική απόσβεση με μητρώο δυναμικής απόσβεσης στο επίπεδο του πεπερασμένου στοιχείου όπως έχει υπολογιστεί στη θεωρία $[C_e]=a_1[K_e]$.

3. Να επαναλάβετε το ερώτημα 2 όταν $\alpha_1=0.05\%$ (αναλογική απόσβεση)
4. Να βρεθούν οι πόλοι και να υπολογιστούν τα δεξιά και αριστερά ιδιοδιανύσματα. Χρησιμοποιώντας αυτά να παρουσιάσετε την απόκριση στο χρόνο για το αποσβενημένο σύστημα.
5. Να βρείτε την μεταβατική απόκριση του συστήματος για τη διέγερση του ερωτήματος 3α χρησιμοποιώντας την άμεση μέθοδο άμεσης ολοκλήρωσης των κεντρικών διαφορών (explicit) και να εξετάσετε την συμπεριφορά της λύσης για μεγάλα και μικρά χρονικά βήματα ολοκλήρωσης, $\Delta t_1=T_1/5$ και $\Delta t_2=T_{\max}/5$ αντίστοιχα. T_1 και $T_{\max}$ είναι οι περίοδοι της χαμηλότερης και υψηλότερης φυσικής συχνότητας του συστήματος.
6. Να επαναλάβετε την αριθμητική ολοκλήρωση χρησιμοποιώντας αντί για τα πλήρη τα μορφικά μητρώα που υπολογίσατε στο ερώτημα 3. Συγκρίνετε τις δύο αποκρίσεις και αναπτύξτε τις παρατηρήσεις σας.

Να λάβετε υπόψη ότι οι αρχικές συνθήκες μετατόπισης, ταχύτητας και επιτάχυνσης είναι μηδενικές

Δίνονται ότι:

$$K_e = \begin{bmatrix} \frac{G \cdot A}{L_e} & 0 & -\frac{G \cdot A}{2} & -\frac{G \cdot A}{L_e} & 0 & -\frac{G \cdot A}{2} \\ 0 & \frac{G \cdot J}{L_e} & 0 & 0 & -\frac{G \cdot J}{L_e} & 0 \\ -\frac{G \cdot A}{2} & 0 & \frac{E \cdot I_{yy}}{L_e} + \frac{G \cdot A \cdot L_e}{3} & \frac{G \cdot A}{2} & 0 & -\frac{E \cdot I_{yy}}{L_e} + \frac{G \cdot A \cdot L_e}{6} \\ -\frac{G \cdot A}{L_e} & 0 & \frac{G \cdot A}{2} & \frac{G \cdot A}{L_e} & 0 & \frac{G \cdot A}{2} \\ 0 & -\frac{G \cdot J}{L_e} & 0 & 0 & \frac{G \cdot J}{L_e} & 0 \\ -\frac{G \cdot A}{2} & 0 & -\frac{E \cdot I_{yy}}{L_e} + \frac{G \cdot A \cdot L_e}{6} & \frac{G \cdot A}{2} & 0 & \frac{E \cdot I_{yy}}{L_e} + \frac{G \cdot A \cdot L_e}{3} \end{bmatrix}$$

$$J = c_2 \cdot h^3 \cdot b$$

$$c_2 = -0.2 \cdot \left(\frac{b}{h}\right)^{-0.8784} + 0.3386$$

$$M_e = \rho \cdot \begin{bmatrix} \frac{A \cdot L_e}{3} & 0 & 0 & \frac{A \cdot L_e}{6} & 0 & 0 \\ 0 & \frac{I_{pp} \cdot L_e}{3} & 0 & 0 & \frac{I_{pp} \cdot L_e}{6} & 0 \\ 0 & 0 & \frac{I_{yy} \cdot L_e}{3} & 0 & 0 & \frac{I_{yy} \cdot L_e}{6} \\ \frac{A \cdot L_e}{6} & 0 & 0 & \frac{A \cdot L_e}{3} & 0 & 0 \\ 0 & \frac{I_{pp} \cdot L_e}{6} & 0 & 0 & \frac{I_{pp} \cdot L_e}{3} & 0 \\ 0 & 0 & \frac{I_{yy} \cdot L_e}{6} & 0 & 0 & \frac{I_{yy} \cdot L_e}{3} \end{bmatrix}$$

Για την 2^η περίπτωση να θεωρήσετε ότι η δυσκαμψία της πτέρυγας είναι συγκεντρωμένη στη δοκό με προφίλ Ι. Συγκεκριμένα εδώ κάνουμε την ακόλουθη παραδοχή, ότι τα αεροδυναμικά φορτία υπολογίζονται για την αεροτομή ενώ το δομικό κομμάτι που παραλαμβάνει το φορτίο είναι μόνο η δοκός με προφίλ Ι όπως αυτό φαίνεται στην Εικόνα 7.

Διαστάσεις:

1. το συνολικό ύψος της δοκού να είναι όσο και της πτέρυγας που είχατε προσομοιώσει
2. Το πλάτος της δοκού bflange το 10% του πλάτους της πτέρυγας ενώ τα υπόλοιπα bweb και το hweb θα μπορούν να είναι οι παράμετροι της μελέτης.

Ταχύτητες:

Για την μελέτη να ξεκινήσετε από ταχύτητα ανέμου ίση με 10m/sec και να αυξάνετε παρατηρώντας τη ταλαντωτική συμπεριφορά της πτέρυγας

Contents

- [Dynamics of aeronautical structures/ Assignment 1 : Dynamics of aircraft wing](#)
- [Given system characteristics](#)
- [Calculation of needed characteristics](#)
- [System definition](#)
- [Finite Element Discretization](#)
- [Task 1 - Compute the natural frequencies and present the first 5 eigenmodes. Ensure convergence for the first 10 eigenfrequencies](#)
- [Task 2 - Compute the equivalent dynamic system from the superposition of eigenmodes. \(1st mode / first 3 modes\). Compute the forced vibration at the free end of the wing.](#)
- [Task 03 - Damping \$C_e=0.05*\[K_e\]\$, compute response for the damped system](#)
- [Task 4 - Compute the poles of the system and the right-left eigenvalues, find the response of the damped system](#)
- [task 05 and task 06 - Numerical integration](#)
- [task 07 - Aeroelasticity](#)
- [Task 07 - I BEAM](#)
- [Element stiffness and mass matrices with Timoshenko assumptions](#)
- [Build the system's Mass matrix and Stiffness matrix](#)
- [find the eigenvalues,natural frequencies and eigenmodes](#)
- [plot eigenmode](#)
- [create the equivalent dynamic system in the modal space. Generalized mass /Generalized stiffness](#)
- [solve the 2nd order ode in the modal space](#)
- [solve 2nd order ode for damped system in modal space](#)

Dynamics of aeronautical structures/ Assignment 1 : Dynamics of aircraft wing

```
% Assignment in the context of "Dynamic of Aeronautical Structures"
% Academic year : 2025-2026
% Department of Mechanical Engineering and Aeronautics
% Theodoros Iliopoulos 1093547- Stylianos Apostolou 1093590

clc

clear
```

Given system characteristics

```
h = 0.15 ; % blade height [m] (y-axis)
L = 8.9 ; % blade length [m] (x-axis)
wi = 2 ; % blade width [m] (z-axis)
```

```
E = 160*1e9 ; % elasticity modulus [Pa]
v = 0.275 ; % poisson ratio [-]
rho = 1578 ; % density [kg/m^3]
```

Calculation of needed characteristics

```
A = wi*h ; % Area
G = E/(2*(1+v)) ; % Shear Modulus [Pa]
Iyy = wi*(h^3)/12 ; % second moment of area around y-axis [m^4]
Izz = h*(wi^3)/12 ; % second moment of area around z-axis [m^4]
Ipp = Iyy + Izz ; % mass moment of inertia [m^4]
c2 = -0.2*((wi/h)^-0.8784)+0.3386; % empirical correction coefficient
J = c2 * (h^3) * wi ; % constant derived from system characteristics
```

System definition

```
system.h = h ;

system.L = L ;

system.w = wi ;

system.E = E ;

system.v = v ;

system.rho = rho ;

system.A = A ;

system.G = G ;

system.Iyy = Iyy ;

system.Izz = Izz ;

system.Ipp = Ipp ;

system.c2 = c2 ;

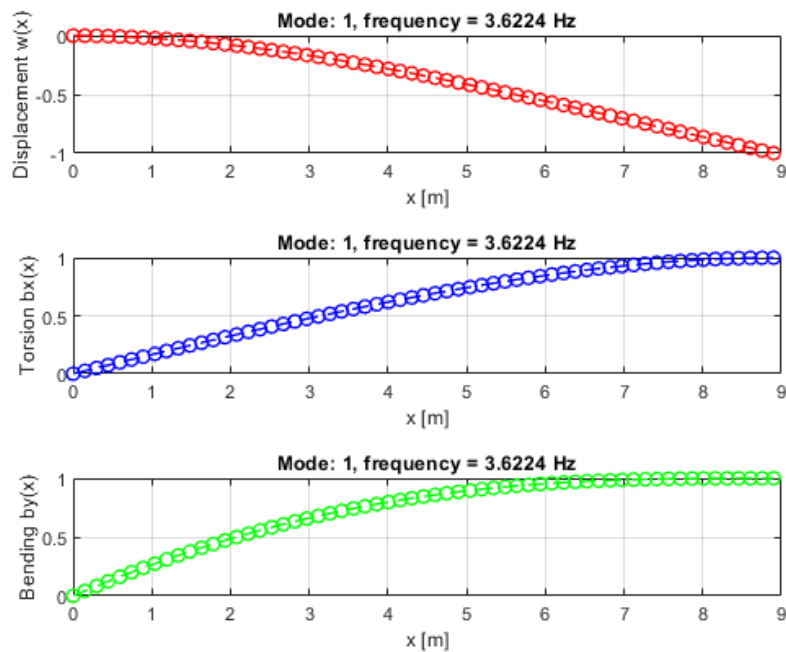
system.J = J ;
```

Finite Element Discretization

```
%-----  
FE.N = 60 ; % number of elements  
%-----  
FE.eff_DOF = 3*FE.N ; % degrees of freedom with boundary conditions applied  
FE.DOF = 3*(FE.N+1) ; % DOF before the boundary conditions  
FE.Le = system.L / FE.N ; % element length (equal length for all FE)
```

**Task 1 - Compute the natural frequencies and present the first 5 eigenmodes.
Ensure convergence for the first 10 eigenfrequencies**

```
[K,M,~] = get_system_matrix(system,FE) ;  
  
[Ke,Me] = get_element_matrix(system,FE) ;  
  
[X,natural_freq,lambda] = solve_eigenvalue_problem(K,M) ;  
  
[w,bx,by]=get_plot(X,natural_freq,system,FE,1) ;
```



Task 2 - Compute the equivalent dynamic system from the superposition of eigenmodes. (1st mode / first 3 modes). Compute the forced vibration at the free end of the wing.

```
modal_M = get_generalized_matrix(X,M) ;

modal_K = get_generalized_matrix(X,K) ;

%-----
% Selection of the number of modes for the superposition
%-----
modes_num = 3 ;
% -----

forcing_omega = 2*pi*(natural_freq(1)+natural_freq(2))/2 ;
forcing_frequency = forcing_omega/(2*pi) ;
%forcing_frequency = 20 ; % for the 2nd case where the engine acts excitation
max_freq= max(forcing_frequency,natural_freq(modes_num)); % max frequency
needed for timestep computation
dt = 0.01/max_freq ; % timestep
total_time = 35*(1/max_freq) ; % amount of periods
t = 0:dt:total_time ;

excitation = sin(forcing_omega*t) ; % initiate only the sinusoidal function
of the excitation, mangitude is found seperately
F0= zeros(FE.eff_DOF,1) ; % magnitude
% apply the force in the correct node
if mod(FE.N,2) == 0
    middle = FE.N/2 ;
else
    middle = (FE.N/2) + 0.5 ;
end

force_acts_on = (middle)*3 -2 ; % applied in the middle, works as an index
F0(force_acts_on,1) = 0.1 ; % magnitude at the proper node -DoF
modal_Fo = X' * F0 ; % equivalent magnitude in the modal space
modal_F = modal_Fo * excitation ; % excitation in the modal space

% [modal_M]q_dot_dot + [modal_K]q = modal_F --> [I]q_dot_dot + [diag(lambda)]q =
Modal_F -reduced problem
% u = X*q (physical space = X * modal space )
q_total_undamped = zeros(modes_num,length(modal_F(1,:))) ; % rows = number of
modes used , columns= discretized time
```

```

q_free_undamped = zeros(modes_num,length(modal_F(1,:))) ; % solution of the
homogeneous - complementary solution
q_forced_undamped = zeros(modes_num,length(modal_F(1,:))) ; % solution of the
non-homogeneous - particular solution

for k = 1:modes_num % find the time-varying functions- modal coefficient of
each mode

    [q_free_undamped(k,:),q_forced_undamped(k,:),q_total_undamped(k,:)] =
solve_DE(t,modal_Fo(k),forcing_frequency,natural_freq(k),true,k) ;

end

u = X(:,1:modes_num) * q_total_undamped ; % the displacement field in the
physical space (functions of time)

% plot tip displacement
figure ;
plot(t(1:1:end),u(end-2,1:1:end),"LineStyle","-","Color","green","LineWidth",1)
grid on ;
xlabel("Time [seconds]")
ylabel("Displacement W [meters]")
title(sprintf("Response (W) for forced vibrarion (f =
%.4f)",(forcing_frequency)))

% ##### GIF - only for testing purposes/crosscheck . ! Frames are
% notequivalent to real time so response shown is not in real time but
% governed by fps . !! It automatically saves it upon run time , dont run it
for long periods, it needs memory
% close all
% filename = 'giffy.gif';
% w_help = u(1:3:end-2,:) ;
% w_actual = [ zeros(1,size(w_help,2)) ; w_help] ;
% min_w = min(w_actual(:)) ;
% max_w = max(w_actual(:)) ;
% sf_gap = 0.15 * (max_w - min_w + eps);
%
% x_points = 0:FE.Le:system.L;
% x_points = x_points.' ;
% figure;
% set(gcf,'Position',[100 100 900 450]);
%
% for i = 1:1:length(w_actual(1,:))
%

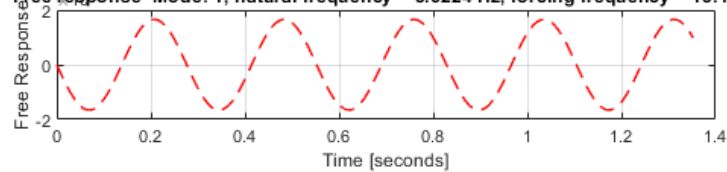
```

```

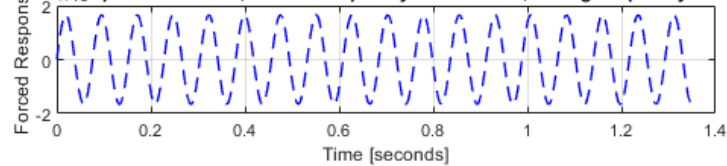
% plot(x_points,w_actual(:,i),'*','LineStyle','-
','Color','red','LineWidth',2,'MarkerSize',8)
% grid on ;
% xlabel('X [m]')
% ylabel(' W(x) [m]')
% title(sprintf("Wing deformation at t = %.4f seconds , forcing frequency =
%.3f Hz",t(i),forcing_frequency))
%
% axis([min(x_points) max(x_points) (min_w-sf_gap) (max_w+sf_gap)]); %
fixed axes
% drawnow;
% % Capture and write frame to GIF
% frame = getframe(gcf);
% im = frame2im(frame);
% [A,map] = rgb2ind(im,256);
%
% if i == 1
%     imwrite(A,map,filename,'gif','LoopCount',Inf,'DelayTime',0.04);
% else
%     imwrite(A,map,filename,'gif','WriteMode','append','DelayTime',0.04);
% end
% end
%
% disp(['Saved GIF: ', filename]);

```

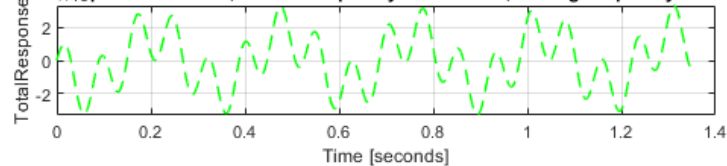
Free response -Mode: 1, natural frequency = 3.6224 Hz, forcing frequency = 13.1522



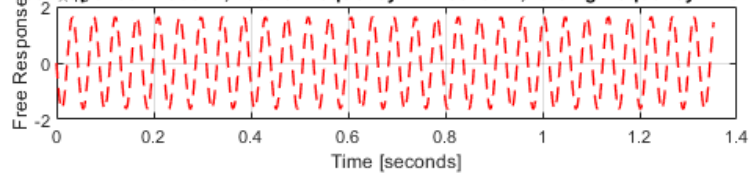
Forced Response -Mode: 1, natural frequency = 3.6224 Hz, forcing frequency = 13.1522



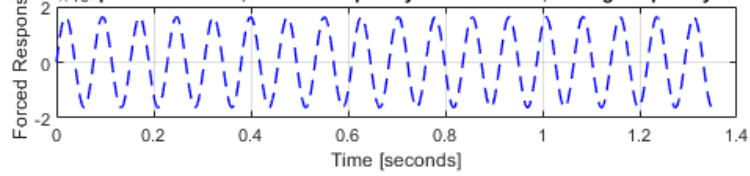
Total Response- Mode: 1, natural frequency = 3.6224 Hz, forcing frequency = 13.1522



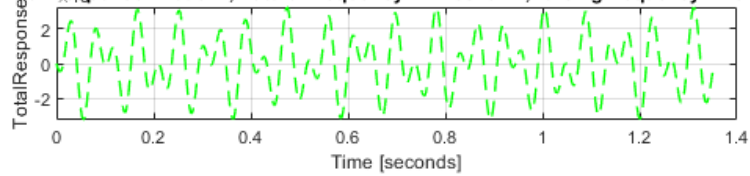
Free response - Mode: 2, natural frequency = 22.6820 Hz, forcing frequency = 13.1522



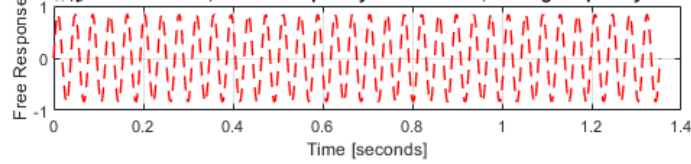
Forced Response - Mode: 2, natural frequency = 22.6820 Hz, forcing frequency = 13.1522



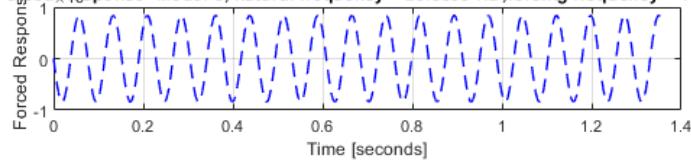
Total Response - Mode: 2, natural frequency = 22.6820 Hz, forcing frequency = 13.1522



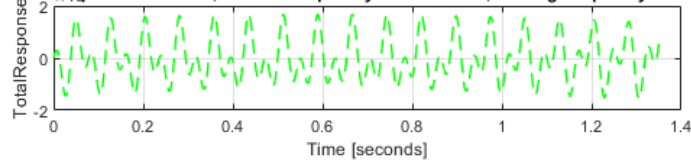
Free response - Mode: 3, natural frequency = 25.8808 Hz, forcing frequency = 13.1522

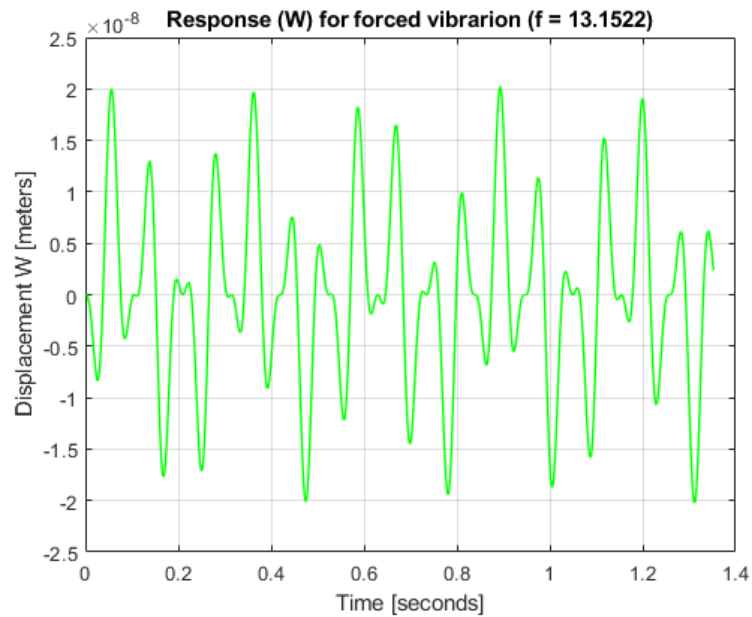


Forced Response - Mode: 3, natural frequency = 25.8808 Hz, forcing frequency = 13.1522



Total Response - Mode: 3, natural frequency = 25.8808 Hz, forcing frequency = 13.1522





Task 03 - Damping $C_e = 0.05 * [K_e]$, compute response for the damped system

```
alpha = 0.0005 ;

damping_ratio = 2*pi*natural_freq * alpha / 2 ;

q_total_damped = zeros(modes_num,length(modal_F(1,:))) ; % rows = number of
modes used , cols= discretized time
q_free_damped = zeros(modes_num,length(modal_F(1,:))) ;
q_forced_damped = zeros(modes_num,length(modal_F(1,:))) ;

for k = 1:modes_num

    [q_free_damped(k,:),q_forced_damped(k,:),q_total_damped(k,:)] =
    solve_DE_damped(t,modal_Fo(k),damping_ratio(k),forcing_frequency,natural_freq(k)
    ),false,k) ;

end

u_damped = X(:,1:modes_num) * q_total_damped ; % the displacement field in the
real space (functions of time)

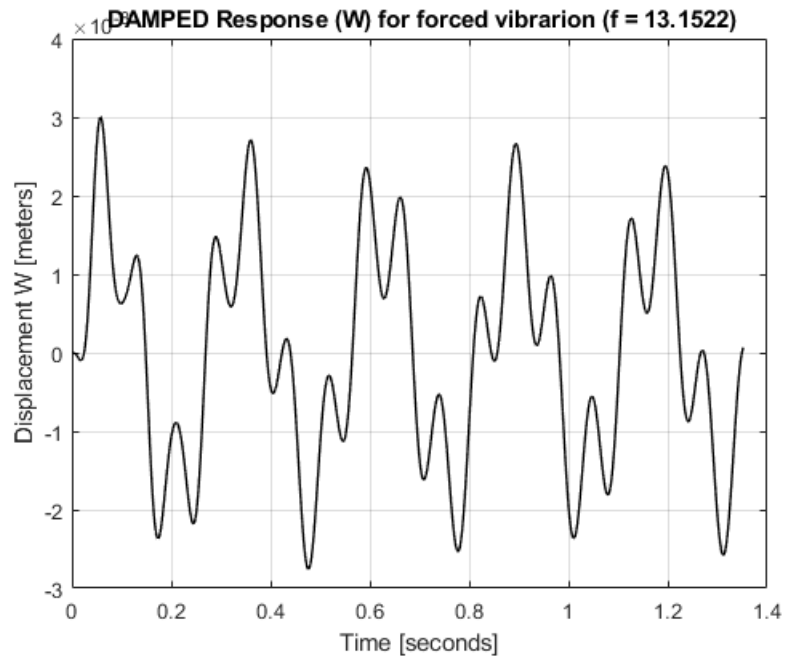
% plots for the damped case
figure ;
plot(t(1:1:end),u_damped(end-2,1:1:end),"LineStyle","-
","Color","k","LineWidth",1)
grid on ;
```

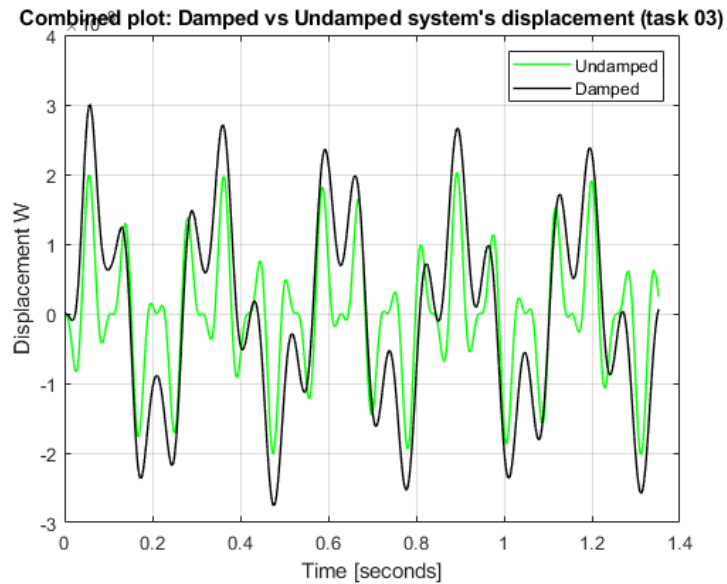
```

xlabel("Time [seconds]")
ylabel("Displacement W [meters]")
title(sprintf(" DAMPED Response (W) for forced vibrarion (f = %.4f)", (forcing_frequency)))

figure;
plot(t(1:1:end),u(end-2,1:1:end),"LineStyle","-","Color","green","LineWidth",1)
hold on;
plot(t(1:1:end),u_damped(end-2,1:1:end),"LineStyle","-","Color","k","LineWidth",1)
grid on ;
ylabel("Displacement W")
xlabel("Time [seconds]")
title("Combined plot: Damped vs Undamped system's displacement (task 03)")
legend('Undamped','Damped') ;

```





Task 4 - Compute the poles of the system and the right-left eigenvalues, find the response of the damped system

```
[~,~,C] = get_system_matrix(system,FE) ;

A_mat_1 = zeros(FE.eff_DOF) ; % DOF x DOF
A_mat_2 = eye(FE.eff_DOF) ;
A_mat_3 = - M \ K ;
A_mat_4 = - M \ C ;
A_mat = [ A_mat_1 , A_mat_2 ; A_mat_3 , A_mat_4] ; % matrix used in the first
order equivalent problem

% B_mat_1 = zeros(FE.eff_DOF,1) ; not needed since forces are not taken into
consideration
% B_mat_2 = inv(M) ;
% B_mat = [B_mat_1 ; (B_mat_2 *F0) ] ;

% solve the right eigenvalue problem; Find right eigenvectors and eigenvalues
[X_right,eig_A] = eig(A_mat) ;

lambda_T4 = diag(eig_A) ; % eigenvalues as a single column , derived from the
normal problem

% solve the adjoint eigenvalue problem; Find the left eigenvectors and
eigenvalues

[X_left,eig_A_adjoint] = eig(A_mat') ;
```

```

lamnda_T4_adjoint = diag(eig_A_adjoint) ;

yes = X_left.' * X_right; % crosscheck that left and right eigenvectors are
orthonormal

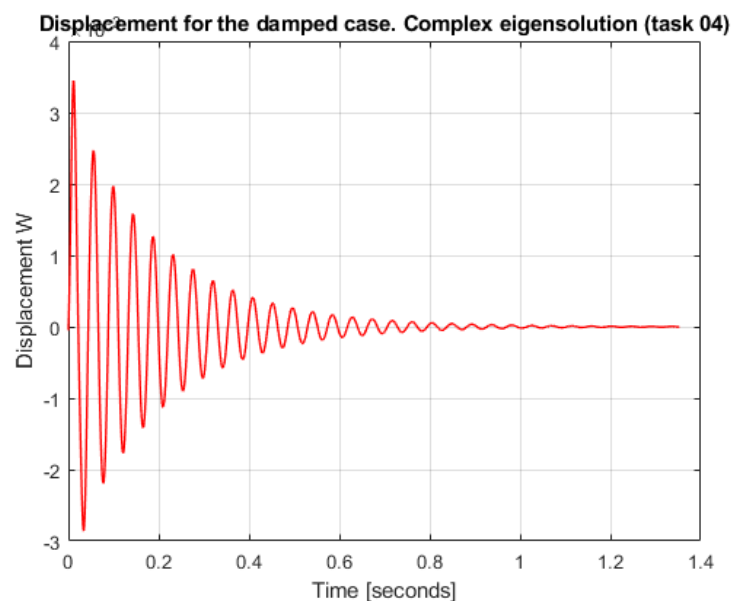
% find the solution as a function of time
x = FE.Le : FE.Le : system.L ;
IC_trial = 0.0001*exp(x) ;
IC = zeros(2*FE.eff_DOF,1) ; % initial condition
IC(1:3:FE.eff_DOF) = IC_trial ;

u_T4 = zeros(2*FE.eff_DOF,length(t)) ; % init solution matrix

for i = 1:length(t)
    u_T4(:,i) = X_right *(diag(exp(lambda_T4*t(i)))) * (X_left.' * IC) ; %
solution
end

% plot tip displacement
figure;
plot(t(1:1:end),u_T4(end-2-FE.eff_DOF,1:1:end),"LineStyle","-
","Color","red","LineWidth",1)
grid on ;
ylabel("Displacement W")
xlabel("Time [seconds]")
title("Displacement for the damped case. Complex eigensolution (task 04)")

```



task 05 and task 06 - Numerical integration

```
tic; % Start timer

dt1 = 1/(5*natural_freq(1)) ; % the timestep is larger than the minimum limit
, the solution breaks
dt2 = 1/(5*natural_freq(end)) ;

total_time_to_end = 50000/natural_freq(end) ; % total sim time
sim_time = 0:dt2:total_time_to_end ; % discretized time
a1 = 1/(dt2^2) ; % auxiliary
a2 = 1/(2*dt2) ; % auxiliary
Fi = F0 * sin(forcing_omega*sim_time) ; % force
modal_Fi = modal_Fo * sin(forcing_omega*sim_time) ; % modal force
modal_C = alpha * modal_K ; % modal damping matrix

x_current = zeros(FE.eff_DOF,1) ; % IC ,
x_previous = zeros(FE.eff_DOF,1) ; % IC
x_current_m = zeros(FE.eff_DOF,1) ; % IC for modal space
x_previous_m = zeros(FE.eff_DOF,1) ; % IC for modal space

x_total = zeros(FE.eff_DOF,length(sim_time)) ; % init solution matrix
x_modal_total = zeros(FE.eff_DOF,length(sim_time)) ; % init modal solution
matrix

%Central differences
for i=1:length(sim_time)

    x_next = (a1*M + a2*C)\( Fi(:,i) - (K - 2*a1*M)*x_current - (a1*M -
a2*C)*x_previous ) ;
    x_modal_next = (a1*modal_M + a2*modal_C)\( modal_Fi(:,i) - (modal_K -
2*a1*modal_M)*x_current_m - (a1*modal_M - a2*modal_C)*x_previous_m ) ;
    x_total(:,i) = x_next ; % update
    x_modal_total(:,i) = x_modal_next ; % update

    if i ~= 1 % special care for the 1st time step ,update
        x_previous = x_total(:,(i-1)) ;
        x_previous_m = x_modal_total(:,(i-1)) ;
    else
        x_previous = zeros(FE.eff_DOF,1) ;
        x_previous_m = zeros(FE.eff_DOF,1) ;
    end
    x_current = x_total(:,i) ;
```

```

    x_current_m = x_modal_total(:,i) ;

end

u_modal_total = X * x_modal_total ; % for modal space, results need to be
multiplied with the modal matrix

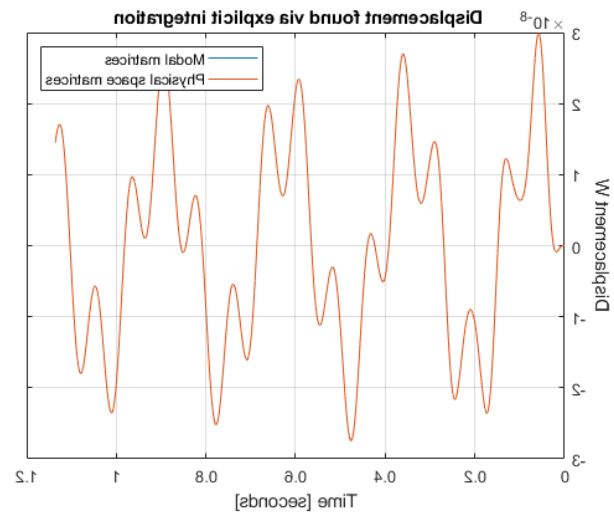
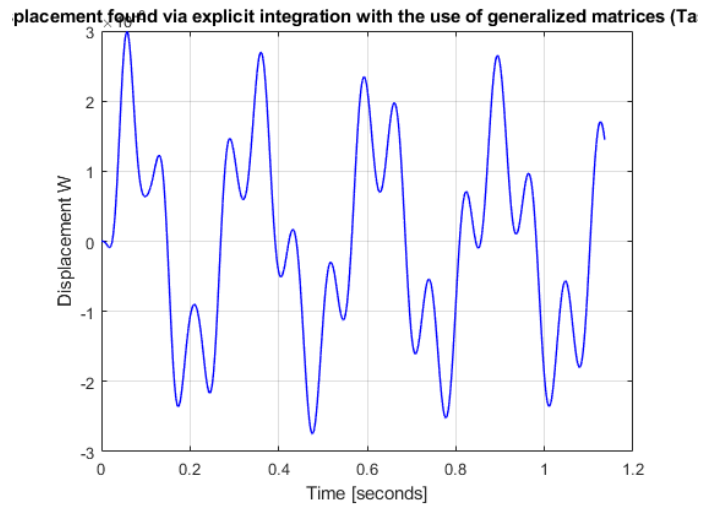
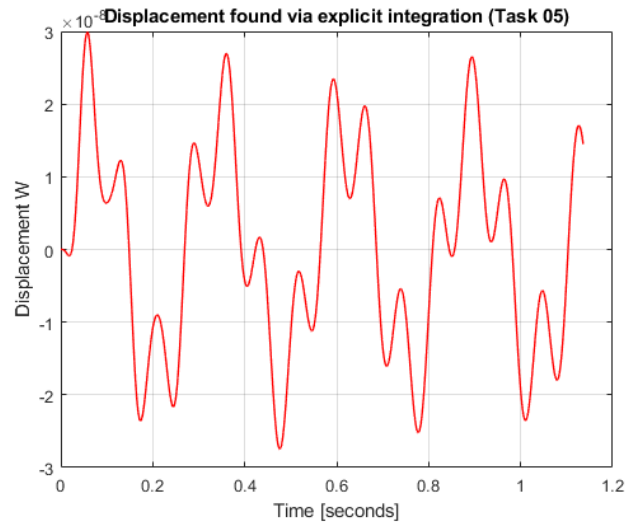
%plot tip displacement
figure;
plot(sim_time(1:1:end),x_total(FE.eff_DOF-2,1:1:end),"LineStyle","-
","Color","red","LineWidth",1)
grid on ;
ylabel("Displacement W")
xlabel("Time [seconds]")
title("Displacement found via explicit integration (Task 05)")

figure;
plot(sim_time(1:1:end),u_modal_total(FE.eff_DOF-2,1:1:end),"LineStyle","-
","Color","blue","LineWidth",1)
grid on ;
ylabel("Displacement W")
xlabel("Time [seconds]")
title("Displacement found via explicit integration with the use of generalized
matrices (Task 06)")

figure;
plot(sim_time(1:1:end),u_modal_total(FE.eff_DOF-
2,1:1:end),sim_time(1:1:end),x_total(FE.eff_DOF-2,1:1:end))
grid on ;
ylabel("Displacement W")
xlabel("Time [seconds]")
title("Displacement found via explicit integration ")
legend("Modal matrices", "Physical space matrices")
elapsedTime = toc; % Stop timer and get elapsed time
fprintf('Elapsed time: %.6f seconds\n', elapsedTime);

```

Elapsed time: 212.540398 seconds



task 07 - Aeroelasticity

```
vel = 20; % air velocity
aoa_zero = 1; % initial angle of attack
a_n = 0.25 ; % newmark a
d_n = 0.5 ; % newmark delta

dt_7 = 0.01; % time step
sim = 0:dt_7:10; % discretized time

% init matrices
R = zeros(FE.eff_DOF,length(sim)) ; % force
u_7 = zeros(FE.eff_DOF,length(sim)) ; % displacement
u_dot_7 = zeros(FE.eff_DOF,length(sim)) ; % velocity
u_dot_dot_7 = zeros(FE.eff_DOF,length(sim)) ; % acceleration

help1 = 1/(a_n*(dt_7^2)) ; % auxiliary
help2 = 1/(a_n*(dt_7)) ; % auxiliary

R(:,1) = Air_load(vel,2,deg2rad(aoa_zero),u_7(:,1),3,(system.w*FE.Le),FE.N) ;
% initial aerodynamic loads
u_dot_dot_7(:,1) = M \ ( R(:,1) - C*u_dot_7(:,1) - K*u_7(:,1) ); % initial
acceleration

% Newmark Method
for i = 1:1:length(sim)-1

    LHS = help1*M + d_n*help2*C + K ; % left hand side
    RHS = R(:,i) + M*(help1*u_7(:,i) + help2*u_dot_7(:,i) + (1/(2*a_n) -
1)*u_dot_dot_7(:,i)) + C*(d_n*help2*u_7(:,i) + ...
((d_n/a_n) -1)*u_dot_7(:,i) + ((d_n/a_n) -
2)*(dt_7/2)*u_dot_dot_7(:,i) ) ; % right hand side

    u_7(:,i+1) = LHS \ RHS ; % next step displacements
    u_dot_dot_7(:,i+1) = help1*(u_7(:,i+1)-u_7(:,i)) - help2*u_dot_7(:,i) -
(1/(2*a_n) -1)*u_dot_dot_7(:,i) ; % next step acceleration
    u_dot_7(:,i+1) = u_dot_7(:,i) + (1-d_n)*dt_7*u_dot_dot_7(:,i)
+d_n*dt_7*u_dot_dot_7(:,i+1) ; % next step velocity

    R(:,i+1) = Air_load(vel,2,deg2rad(1),u_7(:,i+1),3,(system.w*FE.Le),FE.N) ;
% next step aerodynamic loads, calculated based on the displacements last found

end
```

```

% plot
figure('Position', [100, 100, 1200, 400]);

subplot(1,2,1)
plot(sim, u_7(end-2,:), 'LineStyle', '-', 'Color', 'red', 'LineWidth', 1.5)
xlabel('Time [s]')
ylabel('Vertical Displacement [m]')
title('BOX WING - Tip Vertical Displacement')
grid on;

subplot(1,2,2)
plot(sim, u_7(end-1,:), 'LineStyle', '-', 'Color', 'blue', 'LineWidth', 1.5)
xlabel('Time [s]')
ylabel('Rotation  $\beta_x$  [rad]')
title('BOX WING - Tip Rotation')
grid on;

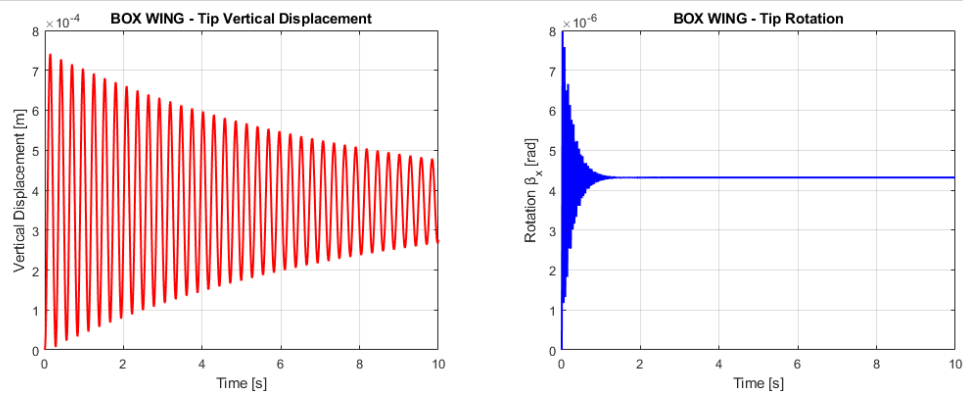
% % ---### similar GIF as in task 02 #####
% close all
% filename = 'giffy.gif';
% w_help = u_7(1:3:end-2,:);
% w_actual = [ zeros(1,size(w_help,2)) ; w_help] ;
% min_w = min(w_actual(:)) ;
% max_w = max(w_actual(:)) ;
% sf_gap = 0.15 * (max_w - min_w + eps);
%
% x_points = 0:FE.Le:system.L;
% x_points = x_points.' ;
% figure;
% set(gcf,'Position',[100 100 900 450]);
%
% for i = 1:1:length(w_actual(1,:))
%
%     plot(x_points,w_actual(:,i),'*','LineStyle','-
%     ','Color','red','LineWidth',2,'MarkerSize',8)
%     grid on ;
%     xlabel('X [m]')
%     ylabel('W(x) [m]')
%     title(sprintf("Wing deformation at t = %.4f seconds , forcing frequency =
%     %.3f Hz",t(i),forcing_frequency))
%
%     axis([min(x_points) max(x_points) (min_w-sf_gap) (max_w+sf_gap)]); %
fixed axes
%     drawnow;

```

```

% % Capture and write frame to GIF
% frame = getframe(gcf);
% im = frame2im(frame);
% [A,map] = rgb2ind(im,256);
%
% if i == 1
%     imwrite(A,map,filename,'gif','LoopCount',Inf,'DelayTime',0.04);
% else
%     imwrite(A,map,filename,'gif','WriteMode','append','DelayTime',0.04);
% end
% end
%
% disp(['Saved GIF: ', filename]);

```



Task 07 - I BEAM

```

% I BEAM

% Geometry characteristics
Bflange = 0.1*system.w; % 10% of original width
Bweb = 0.1*Bflange; % 10% of the flange
Hweb = 0.7*system.h; % 70% of total height
Hflange = 0.15*system.h; % 15% of total height (for each flange )

c2_Ibeam = -0.2*((Bflange/system.h)^-0.8784)+0.3386; % empirical correction
coefficient
%J_Ibeam = c2_Ibeam * (system.h^3) * Bflange ; % constant derived from system
characteristics
J_Ibeam = (1/3)*(2*Bflange*Hflange^3 + Hweb*Bweb^3); % constant

d_Ibeam = (Hweb + Hflange)/2; % distance of flange-web centroids

Iyy_Ibeam = ( Bflange*system.h^3 - (Bflange-Bweb)*(system.h - 2*Hflange)^3)/12 ;

```

```

Izz_Ibeam = ( 2*Bflange*Bflange^3 + Hweb*Bweb^3 )/12 ;

A_Ibeam = 2*Bflange*Hflange + Hweb*Bweb ; % area
Ipp_Ibeam = Iyy_Ibeam + Izz_Ibeam ; % polar moment of inertia

% init characteristics in a struct in order to properly input it in the
functions requirring a struct input
beam.Bflange = Bflange ;
beam.Hflange = Hflange ;
beam.Bweb = Bweb ;
beam.Hweb = Hweb ;
beam.E = E ;
beam.v = v ;
beam.rho = rho ;
beam.A = A_Ibeam ;
beam.G = G ;
beam.Iyy = Iyy_Ibeam ;
beam.Izz = Izz_Ibeam ;
beam.Ipp = Ipp_Ibeam ;
beam.c2 = c2_Ibeam ;
beam.J = J_Ibeam ;

% create the new stiffness,mass,damping matrices
[K_I,M_I,C_I] = get_system_matrix(beam,FE) ;
[Ke_I,Me_I] = get_element_matrix(beam,FE) ;

dt_7_I = 0.01; % time step
sim_I = 0:dt_7_I:10; % discretized time step

% init matrices
R_I = zeros(FE.eff_DOF,length(sim_I)) ;
u_7_I = zeros(FE.eff_DOF,length(sim_I)) ;
u_dot_7_I = zeros(FE.eff_DOF,length(sim_I)) ;
u_dot_dot_7_I = zeros(FE.eff_DOF,length(sim_I)) ;

help1 = 1/(a_n*(dt_7_I^2)) ;
help2 = 1/(a_n*(dt_7_I)) ;

R_I(:,1) = Air_load(vel,2,deg2rad(boa_zero),u_7_I(:,1),3,(system.w*FE.Le),FE.N)
; % initial aerodynamic loads
u_dot_dot_7_I(:,1) = M \ ( R(:,1) - C*u_dot_7(:,1) - K*u_7(:,1) ); % initial
acceleration

% Newmarck method

```

```

for i = 1:1:length(sim_I)-1

    LHS_I = help1*M_I + d_n*help2*C_I + K_I ; % left hand side
    RHS_I = R_I(:,i) + M_I*(help1*u_7_I(:,i) + help2*u_dot_7_I(:,i) + (1/(2*a_n)
- 1)*u_dot_dot_7_I(:,i)) + C_I*(d_n*help2*u_7_I(:,i) + ...
        ((d_n/a_n) -1)*u_dot_7_I(:,i) + ((d_n/a_n) -
2)*(dt_7_I/2)*u_dot_dot_7_I(:,i) ) ; % right hand side

    % u_7(:,i+1) = ( help1*M + d_n*help2*C + K ) \ (R(:,i+1) +
M*(help1*u_7(:,i) + help2*u_dot_7(:,i) + (1/(2*a_n) - 1)*u_dot_dot_7(:,i)) +
C*(d_n*help2*u_7(:,i) + ...
        % ((d_n/a_n) -1)*u_dot_7(:,i) + ((d_n/a_n) -
2)*(dt_7/2)*u_dot_dot_7(:,i) ) );

    u_7_I(:,i+1) = LHS_I \ RHS_I ; % next step displacements
    u_dot_dot_7_I(:,i+1) = help1*(u_7_I(:,i+1)-u_7_I(:,i)) -
help2*u_dot_7_I(:,i) - (1/(2*a_n) -1)*u_dot_dot_7_I(:,i) ; % next step
acceleration
    u_dot_7_I(:,i+1) = u_dot_7_I(:,i) + (1-d_n)*dt_7_I*u_dot_dot_7_I(:,i)
+d_n*dt_7_I*u_dot_dot_7_I(:,i+1) ; % next step velocity

    R_I(:,i+1) =
Air_load(vel,2,deg2rad(aoa_zero),u_7_I(:,i+1),3,(system.w*FE.Le),FE.N) ; %
next step aerodynamic loads, calculated based on the displacements last found

end

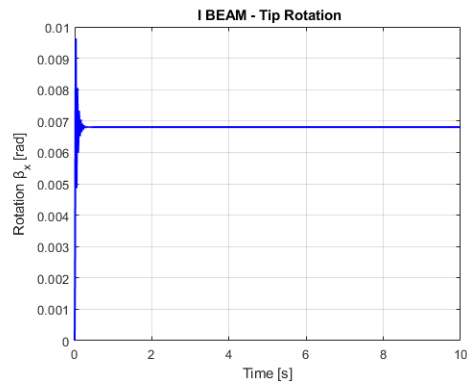
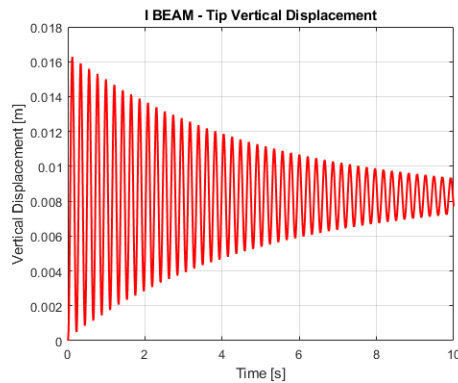
figure('Position', [100, 100, 1200, 400]);

subplot(1,2,1)
plot(sim_I, u_7_I(end-2,:), 'LineStyle', '-', 'Color', 'red', 'LineWidth', 1.5)
xlabel('Time [s]')
ylabel('Vertical Displacement [m]')
title('I BEAM - Tip Vertical Displacement')
grid on;

subplot(1,2,2)
plot(sim_I, u_7_I(end-1,:), 'LineStyle', '-', 'Color', 'blue', 'LineWidth',
1.5)
xlabel('Time [s]')
ylabel('Rotation  $\beta_x$  [rad]')
title('I BEAM - Tip Rotation')

```

```
grid on;
```



Element stiffness and mass matrices with Timoshenko assumptions

```
% Inputs
% -> system : A struct containing all characteristics needed to build the
% elemental matrices such as G , E, J, c2, A, rho etc
% -> FE : A struct containing all the information regarding the FE
% discretization including element number,length, dofs etc
% Outputs:
% -> Ke, Me : elemental matrices

function [Ke,Me] = get_element_matrix(system,FE)

G = system.G ;
A = system.A ;
Le = FE.Le ;
J = system.J ;
E = system.E ;
Iyy = system.Iyy ;
rho = system.rho ;
Ipp = system.Ipp;

% element stiffness matrix
Ke = [G*A/Le 0 -G*A/2 -G*A/Le 0 -G*A/2
      0 G*J/Le 0 0 -G*J/Le 0
      -G*A/2 0 (E*Iyy/Le + G*A*Le/3) G*A/2 0 (-E*Iyy/Le + G*A*Le/6)
      -G*A/Le 0 G*A/2 G*A/Le 0 G*A/2
      0 -G*J/Le 0 0 G*J/Le 0
      -G*A/2 0 (-E*Iyy/Le + G*A*Le/6) G*A/2 0 (E*Iyy/Le + G*A*Le/3)] ;

% element mass matrix (consistent mass approach)
Me = rho*[A*Le/3 0 0 A*Le/6 0 0
          0 Ipp*Le/3 0 0 Ipp*Le/6 0
```

```

    0 0 Iyy*Le/3 0 0 Iyy*Le/6
    A*Le/6 0 0 A*Le/3 0 0
    0 Ipp*Le/6 0 0 Ipp*Le/3 0
    0 0 Iyy*Le/6 0 0 Iyy*Le/3] ;

end

```

Build the system's Mass matrix and Stiffness matrix

```

% Inputs : Same as the previous function
% Output : K, C, M matrices for the whole system
% ** The C matrix is found for proportional damping assumption

function [K_total,M_total,C_total] = get_system_matrix(system,FE)

K_total = zeros(FE.DOF) ;
M_total = zeros(FE.DOF) ;
C_total = zeros(FE.DOF);

for element = 1:FE.N
    i = element ;

    [Ke,Me] = get_element_matrix(system,FE) ;
    %map DOFs
    map_start = 3*(i-1) + 1 ; % current element's 1st DOF position in the total
    matrices, effective DOF start position
    map_end = 3*(i) + 3 ; % current element's 6th DOF position in the total
    matrices, effective DOF end position

    K_total(map_start:map_end,map_start:map_end) =
    K_total(map_start:map_end,map_start:map_end) + Ke ;
    M_total(map_start:map_end,map_start:map_end) =
    M_total(map_start:map_end,map_start:map_end) + Me ;
    C_total(map_start:map_end,map_start:map_end) =
    C_total(map_start:map_end,map_start:map_end) + 0.0005*Ke ;
end
% exclude the 1st node (due to boundary conditions)
K_total(1:3,:) = [] ;
K_total(:,1:3) = [] ;

M_total(1:3,:) = [] ;

```

```

M_total(:,1:3) = [] ;

C_total(1:3,:) = [] ;
C_total(:,1:3) = [] ;

end

```

find the eigenvalues,natural frequencies and eigenmodes

```

% Inputs
function [X,natural_freq,lambda] = solve_eigenvalue_problem(K,M)

%[K , M] = get_system_matrix(system,FE) ;
[X,eig_D] = eig(K,M) ; % x = eigenvector matrix(modal matrix) , eig_D =
eigenvalues (diagonal matrix)

lambda = real(diag(eig_D)) ;
omega = sqrt(lambda) ; % rad /sec
natural_freq = omega / (2*pi) ; % Hz
[natural_freq,index] = sort(natural_freq,'ascend') ; %sort frequencies with
ascending order
X = X(:,index) ; % modes are orthonormal to the M matrix

end

```

plot eigenmode

```

% function made for visualizing modes
% inputs:
%   -> X: the mode we want to visualize
%   -> natural_freq: the equivalent natural frequency
%   -> system : struct, same as before
%   -> FE: struct , same as before
%   -> modes_number :signal the mode that is being handled
% Outputs:
%   -> Triple tile layout with modes for w,bx,by

function [w,bx,by]=get_plot(X,natural_freq,system,FE,modes_number)

w = X(1:3:FE.eff_DOF,:) ;
bx = X(2:3:FE.eff_DOF,:) ;
by = X(3:3:FE.eff_DOF,:) ;

```

```

x_points = 0:FE.Le:system.L;
x_points = x_points.' ;

for i=1:modes_number
    norm_w = w(:,i) / max(abs(w(:,i))) ;
    norm_bx = bx(:,i) / max(abs(bx(:,i))) ;
    norm_by = by(:,i) / max(abs(by(:,i))) ;

    eff_w = [0;norm_w] ;
    eff_bx = [0;norm_bx];
    eff_by = [0;norm_by];

    figure;
    tiledlayout(3,1)
    % w(x) - x
    nexttile
    plot(x_points, eff_w, 'o--', 'Color','red', 'LineWidth',1)
    grid on ;
    xlabel('x [m]')
    ylabel('Displacement w(x)')
    title(sprintf('Mode: %d, frequency = %.4f Hz',i,natural_freq(i)))

    % bx(x) - x
    nexttile
    plot(x_points, eff_bx, 'o--', 'Color','blue', 'LineWidth',1)
    grid on;
    xlabel('x [m]')
    ylabel('Torsion bx(x)')
    title(sprintf('Mode: %d, frequency = %.4f Hz',i,natural_freq(i)))

    % by(x) - x
    nexttile
    plot(x_points, eff_by, 'o--', 'Color','green', 'LineWidth',1)
    grid on;
    xlabel('x [m]')
    ylabel('Bending by(x)')
    title(sprintf('Mode: %d, frequency = %.4f Hz',i,natural_freq(i)))

end

end

```

create the equivalent dynamic system in the modal space. Generalized mass
/Generalized stiffness

```
% Generalized matrix creation
% Inputs:
%   -> eigenmodes : The modal matrix
%   -> matrix: the matrix for which we want to compute the modal equivalent
%   e.g. matrix = M -> finds the modal mass matrix
% Outputs:
%   -> generalized matrix

function [modal_matrix] = get_generalized_matrix(eigenmodes,matrix)

modal_matrix = eigenmodes'* matrix * eigenmodes ; % generalized matrix (modes
are orthonomral to the mass matrix )
% generalized stiffness matrix which equals to the eigenvalues diagonal matrix

% modal_M and modal_K have, effectively, elements only in the diagonal. The off
diagonal elements exist due to roundoff error.
% generalized mass matrix is [I] and generalized stiffness is [lambda] diagonal

end
```

solve the 2nd order ode in the modal space

```
% ODE Solver for the case of undamped reduced system (in modal space)
%   Inputs:
%       -> t: time vector
%       -> modal_F_magntiude : the vector representing the force magnitude in
modal space
%       -> forcing_frequency : the frequency of the excitation
%       -> natural_frequency : the natural frequency for the mode we solve
%       -> show_plot : type Bool, true for showing plot, false for no plot
%       -> mode_flag : index for the mode being handled
%   Output:
%       -> complementary response (solution of homogeneous)
%       -> particular responser
%       -> total response ( addition of the above)

function [q_free,q_forced,q_total] =
solve_DE(t,modal_F_magnitude,forcing_frequency,natural_frequency,show_plot,mode_
flag)
```

```

%solve an undamped forced vibration with harmonic excitation

% zero initial conditions
q0 = 0 ;
q0_dot = 0 ;
natural_omega = natural_frequency * 2*pi ; % rad/sec
forcing_omega = forcing_frequency * 2*pi ; % rad/sec

C = modal_F_magnitude/((natural_omega^2 - forcing_omega^2)) ; % coeff
A = q0 ; % coeff
B = q0_dot/natural_frequency - C; % coeff

q_free = A*cos(natural_omega*t) + B*sin(natural_omega*t) ;
q_forced = C*sin(forcing_omega*t);
q_total = q_free + q_forced ;

% plots
step = 1 ;
if show_plot == true
    figure;
    tiledlayout(3,1)

    nexttile
    plot(t(1:step:end),q_free(1:step:end),'--','Color','red','LineWidth',1)
    grid on ;
    xlabel('Time [seconds]')
    ylabel(' Free Response ')
    title(sprintf('Free response -Mode: %d, natural frequency = %.4f Hz,
forcing frequency = %.4f',mode_flag,natural_frequency,forcing_frequency))

    nexttile
    plot(t(1:step:end),q_forced(1:step:end),'--','Color','blue','LineWidth',1)
    grid on ;
    xlabel('Time [seconds]')
    ylabel('Forced Response')
    title(sprintf(' Forced Response -Mode: %d, natural frequency = %.4f Hz
,forcing frequency = %.4f',mode_flag,natural_frequency,forcing_frequency))

    nexttile
    plot(t(1:step:end),q_total(1:step:end),'--','Color','green','LineWidth',1)
    plot(t(1:step:end),q_total(1:step:end),'--','Color','green','LineWidth',1)
    grid on ;
    xlabel('Time [seconds]')
    ylabel('TotalResponse' )

```

```

        title(sprintf('Total Response- Mode: %d, natural frequency = %.4f Hz,
forcing frequency = %.4f',mode_flag,natural_frequency,forcing_frequency))
    end

end

```

solve 2nd order ode for damped system in modal space

```

% 2nd order ODE solver for damped system. Inputs and outputs are similar to
% the previous function

function [q_free,q_forced,q_total] =
solve_DE_damped(t,modal_F_magnitude,damping_ratio,forcing_frequency,natural_freq
uency,show_plot,mode_flag)

%solve an undamed forced vibration with harmonic excitation

% zero initial conditions
q0 = 0 ;
q0_dot = 0 ;
natural_omega = natural_frequency * 2*pi ;
forcing_omega = forcing_frequency * 2*pi ;
damping_omega = natural_omega*sqrt(1-damping_ratio^2) ;

helper = (forcing_omega/natural_omega)^2 ; % auxiliary

C = (modal_F_magnitude / natural_omega^2) * ( (1- helper)/((1-helper)^2
+(2*damping_ratio*(sqrt(helper)))^2 ) ) ;
D = (modal_F_magnitude / natural_omega^2) * ( (-
2*damping_ratio*sqrt(helper))/((1-helper)^2 +(2*damping_ratio*(sqrt(helper)))^2
) ) ;
A = -D ;
B = (damping_ratio*natural_omega*A - C*forcing_omega )/damping_omega;

q_free = C*sin(forcing_omega*t) + D*cos(forcing_omega*t) ; % particular
q_forced = exp(-damping_ratio*natural_omega*t) .* ( A*cos(damping_omega*t) +
B*sin(damping_omega*t) ) ; % complementary
q_total = q_free + q_forced ; % actual response

% plots
step = 1 ;
if show_plot == true
    figure;
    tiledlayout(3,1)

```

```

nexttile
plot(t(1:step:end),q_free(1:step:end),'--','Color','red','LineWidth',1)
grid on ;
xlabel('Time [seconds]')
ylabel(' Free Response ')
title(sprintf(' (DAMPED)Free response -Mode: %d, natural frequency = %.4f
Hz, forcing frequency = %.4f',mode_flag,natural_frequency,forcing_frequency))

nexttile
plot(t(1:step:end),q_forced(1:step:end),'--','Color','blue','LineWidth',1)
grid on ;
xlabel('Time [seconds]')
ylabel('Forced Response')
title(sprintf(' (Damped)Forced Response -Mode: %d, natural frequency = %.4f
Hz ,forcing frequency = %.4f',mode_flag,natural_frequency,forcing_frequency))

nexttile
plot(t(1:step:end),q_total(1:step:end),'--','Color','green','LineWidth',1)
grid on ;
xlabel('Time [seconds]')
ylabel('TotalResponse' )
title(sprintf(' (Damped)Total Response- Mode: %d, natural frequency = %.4f
Hz, forcing frequency = %.4f',mode_flag,natural_frequency,forcing_frequency))

end

end

```